Transfer Learning for Joint Trajectory Control and Task Offloading in Large-Scale Partially Observable UAV-Assisted MEC

Zhen Gao, Gang Wang, Lei Yang, and Yu Dai

***Abstract*—Existing joint trajectory control and task offloading (JTCTO) algorithms offer ultra-low latency services for smart devices (SDs) in uncrewed aerial vehicle (UAV)-assisted mobile edge computing (MEC). However, these JTCTO algorithms typically require large training datasets to learn the optimal policies, leading to low learning efficiency. Additionally, most existing JTCTO algorithms are difficult to scale to environments with more than a few UAVs, as their complexity increases exponentially with the number of UAVs. In this paper, we propose a decentralized JTCTO algorithm based on the Policy Transfer and Mean Field-based Multi-Agent Actor-Critic (PTMF-MAAC). First, a novel policy transfer algorithm is proposed to determine which UAV's JTCTO strategy is helpful for each UAV and when to terminate the strategy to accelerate the learning efficiency of the UAV. Second, we propose a partially observable mean field algorithm that significantly reduces the model space by replacing the influence of all other UAVs on a particular UAV with an average value, thereby adapting to large-scale UAV scenarios. Experiments have shown that compared to the baseline, PTMF-MAAC reduces the system cost by 18.44%∼28.57% and improves the model learning efficiency and adaptability to partially observable large-scale UAV-assisted MEC.**

***Index Terms*—UAV-assisted MEC, joint trajectory control and task offloading, multi-agent actor-critic, transfer learning, mean field algorithm, partial observation.**

I. Introduction

With the continuous advancement of mobile applications, including intelligent camera surveillance systems and smart driver assistance, an increasing number of tasks require substantial computational resources while being highly sensitive to latency [1], [2]. However, smart devices (SDs) often have limited computing power and battery capacity, making it challenging to meet these demands. As a promising solution, multi-access edge computing (MEC) [3], [4] holds great potential for addressing this challenge. MEC enables SDs to offload their computation-intensive tasks to nearby edge servers (ESs), thereby reducing task processing latency and energy consumption.

Nevertheless, SDs still faces challenges in obtaining dependable computing and communication services in specific exceptional scenarios. For example, in regions prone to natural disasters, numerous SDs may need to handle computation-intensive applications in distant or mountainous locations, where conventional MEC servers might be damaged, and communication conditions could be exceedingly unstable. Fortunately, due to their flexible deployment and broad coverage capabilities, uncrewed aerial vehicles (UAVs) have been leveraged to support MEC systems in handling computation-intensive tasks.

While earlier studies on UAV-assisted networks predominantly concentrate on communication aspects [5], [6], recent research has started to explore UAV-assisted MEC systems, including trajectory planning [7], [8], resource allocation [9], [10], and task offloading [11], [12], [13]. However, compared to existing studies [11], [12], [13] on UAV-assisted MEC, our system model is more practical and challenging. Specifically, first, we consider a multiple UAV-assisted partially observable MEC system environment, where each UAV can only communicate with other UAVs and ground users within its limited communication range due to constraints on maximum communication distance and maneuverability. Second, unlike these research works [11], [12], [13] that assume an even distribution of ground users, we account for their random distribution in UAV-assisted MEC scenarios, which requires the proposed algorithm to have high robustness and exploration ability. Third, due to the difficulty of UAVs obtaining global information in actual UAV-assisted MEC scenarios, we assume that the training data for the algorithm is only composed of information from itself, other UAVs, and ground users within the communication range. In other words, UAVs rely solely on local information learning strategies but should achieve high performance in terms of global indicators. In contrast, existing studies [11], [12], [13] train models using global data to improve the stability of model convergence. Finally, existing task offloading research [9], [10], [11], [12], [13] in UAV-assisted MEC typically considers a small number of UAVs providing computation offloading services to ground users since task offloading algorithms exhibit exponential complexity with respect to the number of UAVs. However, in many scenarios, such as traffic congestion areas, many UAVs

Received 1 August 2024; revised 8 April 2025; accepted 11 June 2025. Date of publication 16 June 2025; date of current version 3 September 2025. This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grant 62402313. Recommended for acceptance by J. Choi. *(Corresponding author: Gang Wang.)*

Zhen Gao is with the School of Information Science and Technology and Artificial Intelligence, Nanjing Forestry University, Nanjing 210037, China (e-mail: gaozhen610351@163.com).

Gang Wang and Lei Yang are with the School of Computer Science and Engineering, Northeastern University, Shenyang 110819, China (e-mail: 1910636@stu.neu.edu.cn; yanglei@mail.neu.edu.cn).

Yu Dai is with the School of College of Software, Northeastern University, Shenyang 110819, China (e-mail: daiy@swc.neu.edu.cn).

Digital Object Identifier 10.1109/TMC.2025.3579748

1536-1233 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 08,2026 at 04:39:35 UTC from IEEE Xplore. Restrictions apply.

are required to collaborate to provide services for a lot of ground users. Therefore, we consider a UAV-assisted MEC scenario with many UAVs.

Furthermore, UAVs typically take off from different locations to reach designated areas where they provide computation and communication services to SDs. The variation in UAV flight paths leads to diverse channel conditions, resulting in differences in communication delays and energy consumption. Additionally, owing to the limited computing and communication capabilities, the number of tasks collected by UAV will affect the task processing delay and energy consumption. Therefore, it is crucial to collaboratively optimize UAV trajectories, task allocation, and communication resource allocation to minimize task execution latency and energy consumption. Beyond these challenges, the design of efficient joint trajectory control and task offloading (JTCTO) methods must also address the following issues. Specifically,

On the one hand, UAV-assisted MEC is a promising paradigm for providing offloading services to SDs. To adapt to the highly dynamic nature of MEC networks, learning agents are deployed on UAVs to sense environmental changes and adjust JTCTO strategies accordingly. A decentralized intelligence framework is established for edge service scheduling by leveraging multiple UAVs. Nevertheless, when UAVs independently learn JTCTO strategies from scratch, they consume significant computational resources, incur additional delays, and ultimately compromise offloading efficiency. To address this challenge, several methods have been proposed [14], [15], [16], [17], [18], [19], [20]. These approaches employ policy distillation algorithms to facilitate knowledge transfer between UAVs. Nevertheless, these methods adopt a coarse-grained approach by simply dividing the training process into two broad stages—learning and transfer—without further refining the specific steps within each stage. As a result, these methods overlook potential optimizations at a finer granularity, leading to inefficient training. Moreover, these methods assume that knowledge transfer holds equal importance throughout the entire training process, which is counterintuitive. The need and significance of knowledge transfer likely vary at different training stages. A high-performance JTCTO policy transfer algorithm should be a dynamic adaptation rather than a unified treatment. For example, the transfer should occur more often at the start of the training since UAVs have less familiarity with the UAV-assisted MEC environment, while it should decrease as the training progresses, as UAVs gradually become more acquainted with the UAV-assisted MEC environment and should concentrate more on their own experiences.

In this paper, we propose a JTCTO method based on a novel JTCTO policy transfer (PT) algorithm, where the JTCTO policy transfer among multiple UAVs is converted into an option learning problem to address. In comparison to the policy distillation algorithm-based JTCTO method [14], [15], [16], [17], [18], [19], [20], our approach is adaptive and suitable for UAV-assisted MEC environments involving more than two UAVs. Specifically, first, our algorithm dynamically chooses the most appropriate JTCTO policy for each UAV to leverage. At the same time, other UAVs will imitate this JTCTO policy and apply it as an additional optimization goal. Second, our algorithm employs termination probability as a performance metric to ascertain whether the utilization should be ended, thereby preventing negative transfer. Third, due to the partial observability of the UAV-assisted MEC system, the update of the option-value function is based on the local environmental interaction experiences of all UAVs. Nevertheless, in such a scenario, each UAV's interaction experience may be inconsistent, leading to oscillations and inaccuracies in the option-value estimation. Finally, our algorithm proposes a new option learning algorithm (i.e., the successor representation option (SRO) learning algorithm). This algorithm is employed to address this discrepancy by disentangling UAV-assisted MEC environment dynamics from rewards to optimize the option-value function based on each UAV's requirements.

On the other hand, in UAV-assisted MEC, several JTCTO algorithms [21], [22], [23], [24], [25] have been proposed. However, these algorithms typically consider only a limited number of UAVs, as their computational complexity grows exponentially with the number of UAVs. Specifically, these algorithms usually employ multi-agent actor-critic algorithms to handle task offloading and resource allocation problems, treating UAVs as agents. The centralized critic-network needs to take the states and actions of all UAVs as input, which increases the input space exponentially. As the number of UAVs increases, the functions that the critic-network needs to learn become extremely complex, resulting in slow training and difficult convergence. Moreover, during the training process, the strategies of each UAV in the UAV-assisted system environment are constantly updated, which means that the environment is dynamic for a UAV, making it difficult for the algorithm to converge. Therefore, most existing JTCTO schemes face scalability challenges [26], [27] and are usually difficult to adapt to UAV-assisted MEC settings with a large number of UAVs. To address this issue, some algorithms [7], [28] reduce the action space by decomposing significant problems (such as resource allocation) into smaller subproblems. This problem-solving approach enables the algorithm to better adapt to large-scale MEC environments. These works are based on a single-agent reinforcement learning (RL) algorithm, which makes it challenging to capture the cooperation and competition relationship between UAVs, resulting in poor collaboration. Typically, multiple UAVs are required to efficiently cooperate to provide services to SDs, reducing the system cost. Moreover, mean field theory has been applied to solve scalability problems in large-scale UAV-assisted MEC system environments [29], [30], [31]. However, these methods [29], [30], [31] assume that UAVs can access all the information within the UAV-assisted MEC system. Gathering complete system-level information in a real-world MEC system can bring substantial communication overheads.

To address the aforementioned issue, our proposed JTCTO method no longer requires UAVs to directly observe the aggregate state variable in a mean-field update. Instead, our proposed algorithm enables UAVs to make JTCTO decisions by maintaining a belief over the aggregate parameter. Specifically, unlike the methods [29], [30], [31], we modify the update rules to relax two key assumptions: (1) global state availability and (2) exact mean action information for all UAVs. We consider a UAV-assisted MEC environment, where each UAV has a

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 08,2026 at 04:39:35 UTC from IEEE Xplore. Restrictions apply.

TABLE I
THE COMPARISON OF THE EXISTING SOLUTIONS AND PTMF-MAAC METHOD

Methods / Essential features	[8], [25], [32]	[21]–[24]	[9]–[13]	[7], [28]	[29]–[31]	**PTMF-MAAC Method**
Single-UAV or ES MEC Networks	✓					
Multi-UAV multi-MS MEC Networks						✓
Joint Trajectory Control, Task Offloading, and Resource Allocation						✓
Policy Distillation		✓				
A novel option learning-based policy transfer algorithm						✓
Small number of UAVs		✓				
Single-agent RL algorithm				✓		
Global system state availability, Exact mean action information for all UAVs					✓	
Partially observable large-scale UAV-assisted MEC environment, only local observation availability						✓
Global information for Training			✓			
Local information for Training						✓

fixed observation radius and can only perceive other UAVs and SDs within its communication range. The contrast between the current approaches and the PTMF-MAAC based on several key features is presented in Table I. The main contributions of this paper are as follows:

- *The Dynamic JTCTO in Large-Scale UAV assisted MEC System Scenario:* We explore a cooperative JTCTO approach in the multi-UAV multi-ES UAV-assisted MEC environments, where numerous UAVs and ESs jointly handle the offloading tasks of multiple SDs. By formulating the JTCTO optimization problem, we aim to minimize the total system cost (i.e., energy consumption and latency) through joint optimization of UAV trajectories, task allocation, and communication resource allocation policies.
- *PTMF-MAAC Algorithm-based JTCTO Solution:* We propose a decentralized JTCTO algorithm based on the Policy Transfer and Mean Field-based Multi-Agent Actor-Critic (PTMF-MAAC) framework. First, a novel policy transfer (PT) algorithm is proposed to determine which UAV's JTCTO strategy can benefit each UAV and determine when to terminate the JTCTO strategy to enhance the learning efficiency of each UAV. Second, we propose an improved version of the mean-field algorithm (i.e., the partially observable mean field (POMF)), which significantly reduces the model space by replacing the influence of all other UAVs on a particular UAV with an average value, thus adapting to large-scale UAV scenarios. Additionally, POMF relaxes the assumption that the UAV must observe the aggregated state variables in the mean-field update. Instead, it enables UAVs to make JTCTO decisions by maintaining belief in the aggregated parameters, thereby improving algorithm scalability and reducing communication overhead.
- *Performance Evaluation:* PTMF-MAAC incorporates the Centralized Training and Distributed Execution (CTDE) mechanism. Specifically, we assume that the adjacent ES initially trains the PTMF-MAAC model in a centralized way. Then, each UAV conducts distributed JTCTO strategies according to the UAV's local states using the trained PTMF-MAAC. Extensive simulation experiments demonstrate that PTMF-MAAC dramatically decreases system costs compared to existing solutions. Moreover, PTMF-MAAC improves model training efficiency and adaptability to large-scale UAV-assisted MEC system scenarios.

The structure of this paper is as follows. Section II reviews the related works. Section III provides the system model and problem formulation for JTCTO in a UAV-assisted MEC system. Section IV defines the RL-based JTCTO problem and presents the proposed JTCTO scheme (i.e., PTMF-MAAC algorithm) and its training process. In Section V, we assess the performance of the PTMF-MAAC algorithm. Finally, Section VI concludes the paper.

II. RELATED WORKS

In this section, we provide detailed related work on JTCTO in UAV-assisted MEC systems. Specifically, first, we explore a large number of JTCTO methods based on transfer learning. Second, we explore JTCTO methods in large-scale UAV-assisted MEC environments.

A. Transfer Learning Based JTCTO Approaches

In [33], the authors propose an automated and quality-aware client selection framework for efficient federated learning. In [34], the authors propose a client selection and bandwidth allocation solution in wireless federated learning networks. In these algorithms, local models on each smart device must aggregate model parameters to the data center to further optimize the global model. In [14], the authors propose an offline-transfer-online framework for cloud-edge collaborative distributed reinforcement learning. Nevertheless, this approach solely relies on the disparity between two unrestricted value functions as a reward for the student, potentially resulting in instability. Moreover, some intelligent sharing methods enable intelligent sharing between multiple UAVs through policy distillation. For example, in [15], the authors propose a safe actor-critic with

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 08,2026 at 04:39:35 UTC from IEEE Xplore. Restrictions apply.

policy distillation approach for the age of information minimization in UAV-assisted Internet of Things networks. In [16], the authors propose a lightweight deep RL-based profit-aware cooperative offloading solution in UAV-enabled MEC systems. In [17], the authors propose a computation offloading algorithm based on edge intelligence for distributed Internet of UAVs. In [18], the authors propose a transfer learning algorithm-based offloading method in vehicular edge computing. In [19], the authors propose a transfer learning-based distributed intelligence method in UAV-enabled edge computing. In [20], the authors propose a knowledge distillation-driven training framework-based joint model selection, training offloading, and resource allocation method. However, these methods only decompose the training process into learning and transfer stages with a coarser granularity. In addition, these methods also believe that knowledge transfer is equally important throughout the entire model training, which is often unrealistic. A good solution is that the importance of knowledge transfer should be different at different stages.

B. JTCTO Approaches in Large-Scale UAV-Assisted MEC Environment

In [21], the authors proposed a multi-agent deep reinforcement learning solution for joint decoupled user association and trajectory design in full-duplex multi-UAV networks. In [22], the authors proposed a multi-agent DRL-based robust computation offloading and trajectory optimization solution for multi-UAV-assisted MEC. In [23], the authors proposed a deep reinforcement learning-based dynamic trajectory control solution for UAV-assisted mobile edge computing. In [24], the authors proposed an RL-based trajectory design and resource allocation solution for multi-UAV networks. In [25], the authors proposed an evolutionary multi-objective reinforcement learning-based trajectory control and task offloading solution in UAV-assisted mobile edge computing. Although these approaches achieve good offloading performance, they only consider small-scale UAVs (e.g., 1∼6 UAV) providing computational services to ground users in UAV-assisted MEC system scenarios. However, with the development of SDs, large-scale UAVs are often needed in many application scenarios to provide computational assistance to ground users, especially in hotspots [7], [28], [29].

Recently, several studies have been proposed to adapt to large-scale UAV-assisted MEC scenarios. For example, in [7], the authors address the model scalability of the proposed algorithm by decomposing the resource allocation problem into a transmit power optimization problem and a task computation allocation problem. In [28], the authors adapt the proposed algorithm to large-scale MEC scenarios by decomposing the scheduling problem into two layers of subproblems and optimizing them alternately through hierarchical RL. These works are algorithms based on single-agent reinforcement learning. Typically, multiple UAVs are necessary to efficiently collaborate and provide computing services to ground users.

Moreover, mean field theory has been applied to solve scalability problems in large-scale UAV-assisted MEC system environments. For example, in [29], the authors proposed a mean-field-aided MARL solution for resource allocation in vehicular networks. In [30], the authors proposed a mean field game-based waveform precoding design solution for mobile crowd-integrated sensing, communication, and computation systems. In [31], the authors proposed a mean field game-based delay-optimal computation offloading solution in large-scale multi-access edge computing. However, these methods must obtain the global state and exact mean action information for all UAVs, which typically results in an enormous communication burden and poor scalability.

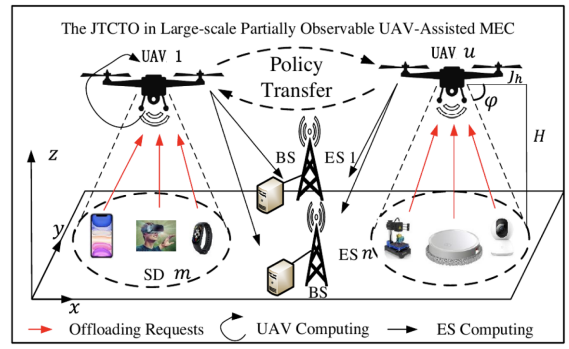

Fig. 1. The large-scale partially observable multi-UAV multi-ES MEC system.

III. System Model

A. System Overview

As illustrated in Fig. 1, we consider an Internet of intelligence-enhanced UAV-assisted MEC system environment. This MEC system contains M SDs, U UAVs, and N ESs. We believe in intelligent applications that periodically generate computationally intensive tasks that the corresponding SDs must handle. Let $J_m(t) = \{d_m(t), \rho_m(t), \lambda_m(t)\}$ represent the task that needs to be processed by the SD m. In this context, $d_m(t)$ signifies the data size of the task, $\rho_m(t)$ indicates the CPU cycle count necessary for task completion, and $\lambda_m(t)$ refers to the task arrival rate. To meet the tasks' latency requirements and reduce SDs' energy consumption, we consider collaborating with UAVs and ESs to handle these computationally intensive tasks. We assume UAVs possess constrained computational and communicational capabilities while ESs have ample resources. In this work, the UAV is responsible for making offloading decisions for task offloading requests from SDs (i.e., partly performed by the UAV and partly performed by the ES). Hence, we take into account four primary elements of the task offloading procedure: 1) The offloaded task transmission from SDs to UAVs. 2) The offloaded task execution at the UAVs. 3) The offloaded task transmission from UAVs to ESs. 4) The offloaded task execution at the ESs. Table II presents key symbols utilized in this paper.

B. UAV Movement Model

We consider that the UAV operates at the height of H, where H represents a fixed positive value. We consider using the direction $\alpha_u(t)$ and distance $l_u(t)$ of the UAV flight at time interval t to represent the trajectory of the UAV u flight. For the flight trajectory $\omega_u(t) = (\alpha_u(t), l_u(t))$ of the UAV u at time

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 08,2026 at 04:39:35 UTC from IEEE Xplore. Restrictions apply.

TABLE II
OVERVIEW OF IMPORTANT VARIABLES

Variables	Descriptions
M	Number of SDs.
N	Number of ESs.
U	Number of UAVs.
$J_m(t)$	The offloaded task.
$d_m(t)$	Data size of the offloaded task.
$\rho_m(t)$	The required CPU cycles of the offloaded task.
$\lambda_m(t)$	The task arrival rate.
$\alpha_u(t)$	UAV's flight angle.
$l_u(t)$	UAV's flight distance.
H	UAV's flight altitude.
$\jmath_h$	UAV's horizontal coverage radius.
$\Phi_m(t)$	Coordinates of SD.
$\Phi_u(t)$	Coordinates of UAV.
φ	Azimuth angle of UAV.
$r_{m,u}(t)$	Data rates between SDs and UAVs.
$\tau^{tr}_{m,u}(t)$	Transmission delay between SDs and UAVs.
$e_{m,u}(t)$	Task energy consumption between SDs and UAVs.
$\tau^{l}_{m,u}(t)$	Task execution delay at the UAVs.
$e^{l}_{m,u}(t)$	Task execution energy consumption at the UAV.
$\chi^{l}_{m,u}(t)$	Task execution proportion at the UAV.
$\chi^{n}_{m,u}(t)$	Task execution proportion at the ES.
$\tau^{l}_{m,u}(t)$	Task execution delay at the UAVs.
$\tau_{m,u,n}(t)$	Task execution delay at the UAVs.

interval t, we have the following constraints,

$$0 \le \alpha_u(t) \le 2\pi, 0 \le l_u(t) \le \jmath_{\max}, \tag{1}$$

where $\jmath_{\max}$ represents the maximum distance that the UAV u can travel in each time interval t. In line with prior research [23], [35], we utilize the Cartesian coordinate system to simulate the action of the UAV. Further, we consider $\Phi_u(t) = [x_u(t), y_u(t), H]^T$ to denote the coordinates of the UAV u. Based on the flight trajectory of the UAV, we can calculate the coordinates of the UAV at the next time interval $t+1$ by using the following (2),

$$\begin{cases} x_u(t+1) = x_u(t) + l_u(t) \cdot \cos(\alpha_u(t)), \\ y_u(t+1) = y_u(t) + l_u(t) \cdot \sin(\alpha_u(t)). \end{cases} \tag{2}$$

In addition, we consider UAVs flying in a fixed area to provide computing services to SDs. Therefore, for the coordinates of the UAV, we have the following constraints,

$$0 \le x_u(t) \le \jmath^x_{\max}, 0 \le y_u(t) \le \jmath^y_{\max}, \tag{3}$$

where $\jmath^x_{\max}, \jmath^y_{\max}$ denote the side lengths of the UAV flight region.

Moreover, in order to avoid collisions between two UAVs, there must be a minimum separation distance $\jmath^{u,u'}_{\min}$ maintained between them. Therefore, we introduce the following collision avoidance constraint,

$$||\Phi_u(t) - \Phi_{u'}(t)|| \ge \jmath^{u,u'}_{\min}, \forall u, u', u \ne u', \tag{4}$$

where $\Phi_{u'}(t) = [x_{u'}(t), y_{u'}(t), H]^T$ denotes the coordinates of the UAV u'.

Besides, we consider that each UAV has its coverage area. Further, we define $\jmath_h$ to denote the horizontal coverage radius of the UAV. Based on the azimuth angle φ, we can compute the horizontal coverage radius $\jmath_h$ of the UAVs through the following (5),

$$\jmath_h = H \cdot \tan(\varphi). \tag{5}$$

C. Offloaded Task Transmission From SDs to UAVs

We consider UAVs and ESs collaboratively handling the offloaded tasks. These tasks are first transmitted to the UAV. We consider the coordinates of the SD m denoted as $\Phi_m(t) = [x_m(t), y_m(t), 0]^T$. Therefore, the distance from UAV u to SD m can be calculated using the (6):

$$\jmath_{u,m}(t) = ||\Phi_u(t) - \Phi_m(t)||. \tag{6}$$

Similar to these works [23], [28], [36], we adopt the orthogonal frequency-division multiple access communication model responsible for the communication between UAVs and SDs in the service range of UAVs. In this scenario, we continue to disregard channel interference among SDs. Due to UAVs' elevated height, the Line of Sight channel is significantly more prevalent than other disturbances like obscuration or minor-scale fading. The Doppler effect resulting from the rapid movement of UAVs is presumed to be fully counterbalanced at the SDs [37]. Consequently, using the free-space path loss formula, we determine the data upload rate between SD m and UAV u through the subsequent (7),

$$r_{m,u}(t) = \frac{W_u}{N^u_m(t)} \log_2\left(1 + \frac{g_0 p_m}{\jmath_{u,m}(t)^2 \cdot \sigma^2_u}\right), \tag{7}$$

where W_u denotes the bandwidth of the UAV u. $N^u_m(t)$ denotes the count of SDs served by the UAV u. g_0 represents the power gain at a baseline distance of 1 meter. p_m signifies the transmission power of SD. σ^2_u represents the cumulative white Gaussian noise strength in every UAV.

Considering that all the offloaded tasks of the SD are uploaded to the UAV, the transmission delay of the tasks between the SD m and the UAV u can be defined through the subsequent (8),

$$\tau^{tr}_{m,u}(t) = \frac{d_m(t)}{r_{m,u}(t)}. \tag{8}$$

Moreover, we specify the energy usage for transmission between the SD m and the UAV u by the following (9),

$$e_{m,u}(t) = p^a_u(t) \cdot \tau^{tr}_{m,u}(t), \tag{9}$$

where $p^a_u(t)$ represents the power received of UAV u.

D. Offloaded Task Execution at the UAVs

Upon receiving offloaded tasks from an SD, the UAV assumes the responsibility of determining the distribution of tasks, deciding what portion is to be executed locally on the UAV, and what portion should be transmitted to the ES for execution. We consider $\chi^l_{m,u}(t)$ to denote the proportion of tasks executed on the UAV u. $\chi^n_{m,u}(t)$ to denote the proportion of tasks executed on the ES n. Therefore, we define the task computation delay on the UAV by the (10),

$$\tau^l_{m,u}(t) = \frac{\chi^l_{m,u}(t) d_m \rho_m(t)}{f^l_{m,u}(t)}, \tag{10}$$

where $\chi^l_{m,u}(t) \in [0,1]$. $f^l_{m,u}(t) = F_{u/} N^u_m(t)$ indicates that SD m are allocated computing resources from UAV u. F_u denotes the computing power possessed by the UAV u. In addition to this, we define the task execution energy consumption on the UAV by the following (11),

$$e^l_{m,u}(t) = \xi (f^l_{m,u}(t))^3 \tau^l_{m,u}(t), \tag{11}$$

where ξ, a non-negative value, represents the efficient switched capacitance.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 08,2026 at 04:39:35 UTC from IEEE Xplore. Restrictions apply.

E. Offloaded Task Transmission From UAVs to ESs

We consider the coordinates of the ES n denoted as $\Phi_n(t) = [x_n(t), y_n(t), 0]^T$. Therefore, the distance between the UAV u and the ES n can be calculated by the following (12),

$$\jmath_{u,n}(t) = ||\Phi_u(t) - \Phi_n(t)||. \tag{12}$$

Similar to the task transmission from SDs to UAVs, we determine the data upload rate between ES n and UAV u through the subsequent (13),

$$r_{u,n}(t) = \frac{W_n}{N_u^n(t)} \log_2 \left(1 + \frac{g_0 p_n(t)}{\jmath_{u,n}(t)^2 \cdot \sigma_n^2}\right), \tag{13}$$

where W_u denotes the bandwidth of the ES n. $N_u^n(t)$ represents the count of UAVs served by the ES n. $p_u(t) \in [0, P_{\max}]$ signifies the transmission power of UAV. $P_{\max}$ denotes the maximum transmission capability of every UAV. σ_n^2 represents the cumulative white Gaussian noise strength in every ES.

Considering that the offloaded tasks need to be transferred to the ES for execution, the transmission delay of the tasks between the UAV u and the ES n can be defined through the subsequent (14),

$$\tau_{m,u,n}^{tr}(t) = \frac{\chi_{m,u}^n(t) d_m(t)}{r_{u,n}(t)}. \tag{14}$$

Moreover, we define the transmission energy consumption between the UAV u and the ES n by the following (15),

$$e_{m,u,n}(t) = p_u(t) \cdot \tau_{m,u,n}^{tr}(t). \tag{15}$$

F. Offloaded Task Execution at the ESs

Upon receiving offloaded tasks from the UAV, ESs commence processing the computational tasks. $\chi_{m,u}^n(t)$ to denote the percentage of tasks executed on the ES n. Therefore, we define the task computation delay on the ES by the following (16),

$$\tau_{m,u,n}(t) = \frac{\chi_{m,u}^n(t) d_m \rho_m(t)}{f_{m,n}(t)}, \tag{16}$$

where $\chi_{m,u}^n(t) \in [0, 1]$. $f_{m,n}^l(t) = F_n / N_m^n(t)$ indicates that SD m are allocated computing resources from ES n. F_n denotes the computing power possessed by the ES n.

IV. Problem Definition and Formulation

A. Problem Definition

Upon the completion of computational tasks for all SDs, the energy consumption of UAV u is determined as

$$E_u^{tot}(t) = \sum_{m=1}^{M} \lambda_m(t) I_{m,u}(t) [e_{m,u}(t) + e_{m,u}^l(t) + e_{m,u,n}(t)], \tag{17}$$

where $I_{m,u}(t)$ indicates whether the SD m is served by the UAV u. If the SD m is served by the UAV u, then $I_{m,u}(t) = 1$, otherwise $I_{m,u}(t) = 0$. Moreover, the execution delay of UAV u is determined as

$$\tau_u^{tot}(t) = \sum_{m=1}^{M} I_{m,u}(t) [\tau_{m,u}^{tr}(t) + \nu], \tag{18}$$

where $\nu = \max_n \{\tau_{m,u}^l(t), \tau_{m,u,n}^{tr}(t) + \tau_{m,u,n}(t)\}$. The max operation indicates that we consider the task of processing at UAV and the task of processing at ES to be simultaneous. The maximum processing delay of these two parts is the processing delay of the task. Further, we consider the system cost of UAV u as a weighted sum of energy consumption and delay expressed through the following (19),

$$C_u(t) = c_1 E_u^{tot}(t) + c_2 \tau_u^{tot}(t), \tag{19}$$

where c_1 and c_2 denote constants.

In this work, we consider minimizing the system cost by optimizing the flight trajectory of the UAV $\omega_u(t)$, the task offloading proportion $(\chi_{m,u}^l(t), \chi_{m,u}^n(t))$, and the transmission power allocation of the UAV $p_u(t)$. Therefore, we formulate the problem by the following (20),

$$\min_{\substack{[\omega_u(t), \chi_{m,u}^l(t), \\ \chi_{m,u}^n(t), p_u(t)]}} \quad \sum_{u=1}^{U} \{C_u(t)\} \tag{20}$$

$$\text{s.t.} \quad C_1 : \chi_{m,u}^l(t) \in [0,1], \chi_{m,u}^n(t) \in [0,1], \tag{20a}$$

$$C_2 : \chi_{m,u}^l(t) + \sum_n \chi_{m,u}^n(t) = 1, \tag{20b}$$

$$C_3 : p_u(t) \in [0, P_{\max}], \tag{20c}$$

$$C_4 : Eq.(1), Eq.(3), Eq.(4), \tag{20d}$$

where the constraint (20a) and constraint (20b) denote the limit of the task offloading ratio. The constraint (20c) indicates that the transmission power allocation of the UAV cannot exceed the maximum power of the UAV. The constraint (20d) indicates the restriction of UAV flight path at each time interval, the restriction of UAV flying out of the predefined area, and the collision constraint between UAVs.

B. MDP-Based Problem Formulation

In our environment, each UAV optimizes its flight trajectory, transmission power allocation, and task offloading ratio to minimize the delay and energy consumption of task processing. Moreover, we redefine the task offloading optimization problem as a multi-agent Markov decision process $\{\mathcal{U}, \mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{P}, \{r_u\}_{u \in \mathcal{U}}, \gamma\}$. Each UAV is regarded as an individual agent within the UAV-assisted MEC system environment, with $\mathcal{U}$ representing the collective set of these UAV agents. $\mathcal{S}$ denotes the system state of the UAV-assisted MEC environment. $\mathcal{O}$ denotes the observation space of the UAV. $\mathcal{A}$ denotes the action space of the UAV. $\mathcal{P}$ denotes state transition probability. r_u denotes the UAV's reward.

1) Local Observation Space $\mathcal{O}$: The local observation of UAV agent u at time slot t is represented as $o_u(t)$, encompassing the following elements: $\Phi_u(t)$, $\Phi_{u,1,..,m}^{\jmath_h}(t)$, and $\Phi_{u,1,..,u'}^{\jmath_h}(t)$, i.e., $o_u(t) = \{\Phi_u(t), \Phi_{u,1,..,m}^{\jmath_h}(t), \Phi_{u,1,..,u'}^{\jmath_h}(t)\} \in \mathcal{O}$. $\Phi_u(t)$ indicates the coordinates of the UAV agent u. $\Phi_{u,1,..,m}^{\jmath_h}(t)$ indicates the coordinates of the SDs within the communication radius $\jmath_h$ of the UAV u. $\Phi_{u,1,..,u'}^{\jmath_h}(t)$ indicates the coordinates of the UAVs within the communication radius $\jmath_h$ of the UAV u. S denotes the system state of the UAV-assisted MEC environment. Further, we have joint observations of all UAVs $S(t) = \{o_1(t), \ldots, o_U(t)\}$.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 08,2026 at 04:39:35 UTC from IEEE Xplore. Restrictions apply.

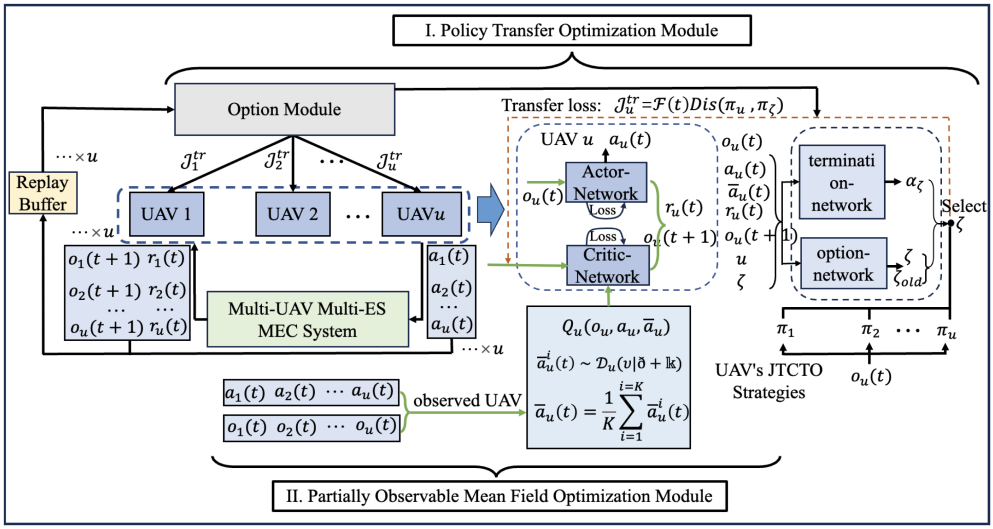

Fig. 2. The PTMF-MAAC algorithm-based JTCTO method in a large-scale partially observable Multi-UAV Multi-ES MEC system.

2) The Action Space $\mathcal{A}$: The action of the UAV agent u is expressed as $a_u(t) = \{\omega_u(t), \chi^l_{m,u}(t), \chi^n_{m,u}(t), p_u(t)\} \in \mathcal{A}$. $\omega_u(t)$ denotes the flight trajectory of the UAV u. $\chi^l_{m,u}(t)$ represents the fraction of tasks offloaded to UAV u for local processing. $\chi^n_{m,u}(t)$ signifies the fraction of tasks offloaded to ES n for processing. $p_u(t)$ is the transmit power of UAV. Further, we have joint actions of all UAVs $A(t) = \{a_1(t), \ldots, a_U(t)\}$.

3) Reward Function $r_u(t)$: To address the optimization problem described in (20), the U UAVs must work together to reduce the overall cost of the system, ensuring compliance with specific requirements and limitations. Then, the UAV's reward function is characterized as the inverse of the system's total cost, provided that all constraints are met. However, in scenarios where certain constraints are not adhered to, appropriate penalties will be integrated into the reward function $\mathcal{R}_u$. Therefore, taking into account the factors above, the reward function for the UAV u is determined as follows,

$$r_u(t) = \begin{cases} -C_u(t), & \text{if meeting the conditions,} \\ -\eta_1 - \eta_2, & \text{otherwise.} \end{cases} \tag{21}$$

where $-\eta_1$ denotes the penalty for collisions between UAVs. $-\eta_2$ denotes the penalty given to the UAV for flying out of the predetermined area.

V. PTMF-MAAC Based JTCTO Method

A. The Overview of the PTMF-MAAC Algorithm

As shown in Fig. 2, we propose a decentralized JTCTO algorithm based on Policy Transfer and Mean Field-based Multi-Agent Actor-Critic (PTMF-MAAC). This algorithm consists of two key modules: a policy transfer optimization module and a partially observable mean field optimization module. Specifically, we first introduce the policy transfer optimization module to enhance model training efficiency. Then, to ensure scalability in UAV-assisted MEC environments with many UAVs, we incorporate the partially observable mean field optimization module. Next, we provide a detailed description of each module.

B. Policy Transfer Optimization Module

As shown in Fig. 2, the policy transfer optimization module consists of two main components, i.e., the interaction of the u UAVs and the multi-UAV multi-ES MEC system and the option-network that determines which UAV's JTCTO strategy is beneficial to the other UAVs. The option-network first initializes the options set $\Theta = \{\zeta_1, \ldots, \zeta_u\}$ by employing u options. And each option consists of three elements: $\zeta_u = \{\mathcal{Y}_{\zeta_u}, \pi_{\zeta_u}, \alpha_{\zeta_u}\}$, where π_{ζ_u} denotes the JTCTO strategy π_u of UAV u. At the beginning of every episode, the option module chooses an option ζ for each UAV according to the option-network and the termination-network. The chosen option remains active until the termination network signals its conclusion, at which point a new option is selected, and the process repeats. During model training, the option module updates the parameters of the option network and the termination network by utilizing experiences gathered from all UAV-MEC interactions. Each UAV then adopts a learned JTCTO strategy from another UAV π_ζ, depending on the chosen option ζ. This process is carried out by JTCTO strategies imitation, which serves as an additional optimization objective (the option module is in charge of this task, and each UAV does not know which JTCTO strategies need to be imitated and how the corresponding loss functions are computed). The exploitation phase concludes when the chosen option ends; subsequently, another option is chosen to iterate the procedure. By enabling UAVs to leverage effective JTCTO strategies from others, the proposed approach accelerates training and enhances the overall learning performance of the algorithm.

Next, we provide a detailed introduction to the combination of the policy transfer optimization module and the Actor-Critic framework. This module integrates with other RL and MARL methods using a similar approach. The training process of the policy transfer optimization module combined with the Actor-Critic framework is shown in Algorithm 1. In this Algorithm 1, we first initialize the replay buffer E_u and the option set

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 08,2026 at 04:39:35 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: The Training Process of the Policy Transfer Optimization Module Combined With Actor-Critic Framework.

Input: Initialize the replay buffer E_u. Initialize the option set $\Theta = \{\zeta_1, \ldots, \zeta_u\}$. Initialize the actor-network and the critic-network with parameter θ_u and parameter φ_u, respectively.

for episode $= 1, \ldots, M$ **do**
  Choose an option ζ for every UAV u.
  Each UAV u chooses a JTCTO strategies $a_u(t) \sim \pi_u(o_u(t))$.
  Execute JTCTO strategies, obtain $r(t)$ and the state of the next time interval.
  Save the interaction experience $(o_u(t), a_u(t), r_u(t), o_u(t+1), \zeta, u)$ in the replay buffer E_u.
  Choose ζ' if ζ ends for each UAV u.
  for each UAV u **do**
    Minimizing the loss of critic-network with respect to θ_u (22)
    the policy transfer optimization module computes policy transfer loss $\mathcal{J}_u^{tr}$.
    Minimizing the loss of actor-network with respect to φ_u (23)
  end for
  Optimize the option-network (**Algorithm 2**).
end for

$\Theta = \{\zeta_1, \ldots, \zeta_u\}$. Additionally, we initialize the actor-network and the critic-network with parameter θ_u and parameter φ_u, respectively. Second, at the beginning of each episode, the option module chooses an option ζ for every UAV u. And each UAV u chooses a JTCTO strategies according to $a_u(t) \sim \pi_u(o_u(t))$. Third, the UAV-assisted MEC environment executes JTCTO strategies $A(t)$, yielding a reward $r(t)$ and transitioning to the next state $S(t+1)$. Finally, we save the interaction experience $(o_u(t), a_u(t), r_u(t), o_u(t+1), \zeta, u)$ into the replay buffer E_u. If the selected option ζ terminates, the option module assigns a new option ζ' for each UAV u, and the process continues.

At each model training step, the UAV u optimizes the critic-network by minimizing the loss function $\mathcal{J}_u^{critic}$, given by

$$\mathcal{J}_u^{critic} = -\sum_{t=1}^{T}\left(\sum_{t'>t}\gamma^{t'-t} r_u(t) - V_{\theta_u}(o_u(t))\right)^2, \tag{22}$$

Furthermore, the UAV u optimizes the actor-network by minimizing the weighted sum of the actor-network own loss and the information sharing loss $\mathcal{J}_u^{sh}$, expressed as

$$\bar{\mathcal{J}}_u^{actor} = \Re_1\left(\sum_{t=1}^{T}\frac{\pi_u(a_u(t)|o_u(t))}{\pi_u^{old}(a_u(t)|o_u(t))}A_u - \lambda KL[\pi_u^{old}|\pi_u]\right), + \Re_2\mathcal{J}_u^{sh}, \tag{23}$$

where $\Re_1$, $\Re_2$ are constants that balance the actor-network's intrinsic loss and the information-sharing loss. $A_u = \sum_{t'>t}\gamma^{t'-t} r_u(t) - V_{\theta_u}(o_u(t))$ denotes the advantage function of the UAV u.

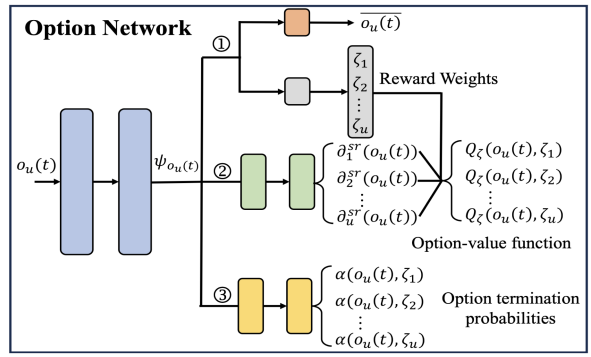

Fig. 3. The successor representation option learning algorithm.

To improve the efficiency of JTCTO strategies transfer between UAVs, we further compute the difference $\mathrm{Dif}(\pi_\zeta, \pi_u)$ between each utilized JTCTO strategies and each UAV's JTCTO strategies and pass this difference to each UAV. Therefore, each UAV can learn another UAV's learned JTCTO strategies π_ζ by minimizing the following loss function $\mathcal{J}_u^{sh}$:

$$\mathcal{J}_u^{tr} = \mathcal{F}(t)\mathrm{Dif}(\pi_\zeta, \pi_u), \tag{24}$$

where the function $\mathcal{F}(t) = 0.5 + \tanh(3 - \sigma t)/2$ represents the discounting factor. σ is a hyper-parameter that dictates the diminishing extent of the weight. At the initial stage of learning, each UAV primarily leverages knowledge from other UAVs. As the learning process progresses, the influence of external knowledge diminishes, and each UAV gradually shifts its focus toward refining its own self-learned policy.

Next, we provide a detailed introduction to the option-network. The primary function of the option-network is to evaluate which UAV's JTCTO strategies are beneficial for each UAV. The structure of the option network is illustrated in Fig. 3, where the local information $o_u(t)$ from the UAV u serves as the input. The option-network further converts the local observation information $o_u(t)$ of the UAV u into the corresponding embedding representation $\psi_{o_u(t)}$ through the fully connected layer and passes it to the three network modules. Specifically, the first module reconstructs the local observation $o_u(t)$ using $\psi_{o_u(t)}$ to enhance its representation and estimates the instantaneous return by employing a linear function $\psi_{o_u(t)} : r_u(\psi_{o_u(t)}) \approx \psi_{o_u(t)} \cdot \varpi$, where $\varpi \in \mathbb{R}^D$ is a weight vector. The second module approximates the value of the option network for different options, denoted as $\partial^{sr}(\psi_{o_u(t)}, \zeta|\varrho)$, which estimates the expected discounted future state occupancy when executing option ζ. The final network module is employed to optimize the termination probability, represented as $\alpha(\psi_{o_{u'}(t)}, \zeta|\hbar)$. Through these modules, the option network effectively identifies and facilitates the transfer of beneficial JTCTO strategies among UAVs.

Based on $Y^{on}(\psi_{o_u(t)}, \zeta|\varrho)$, the value of the option-network can be calculated through $Q_\zeta(\psi_{o_u(t)}, \zeta) \approx \partial^{sr}(\psi_{o_u(t)}, \zeta|\varrho) \cdot \varpi$. Given that options are variables that fluctuate over time, the option-network needs to compute a utility that represents the anticipated discounted future state occupation of carrying out an option ω based on inputting a local state information $o_u(t)'$

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 08,2026 at 04:39:35 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2: The Training Process of the Option-Network.

Input: Initialize the option set $\Theta = \{\zeta_1, \ldots, \zeta_u\}$.
Initialize the replay buffer E_u for each UAV u.
Initialize the local observation feature and the reward weights with parameter ϕ and parameter ϖ, respectively.
Initialize the local observation reconstruction, the termination-network with parameter $\bar{\vartheta}$ and parameter ε, respectively.
Initialize the option-network, the target option-network with parameter ϱ and parameter ϱ', respectively.
for each update step **do**
  Choose the samples $(o_u(t), a_u(t), r_u(t), o_u(t+1), \zeta, u)$ from each E_u.
  Update $\mathcal{J}(\bar{\phi}, \phi)$ with respect to $\bar{\phi}, \phi$ ((26))
  Update $\mathcal{J}(\varpi, \phi)$ with respect to ϖ, ϕ ((26))
  for each ζ **do**
    if π_ζ choose JTCTO strategies $a_u(t)$ based on local observation information $o_u(t)$ **then**
      Compute $\tilde{Y}(\psi_{o_u(t)}, \zeta|\varrho)$ ((25))
      Update $\mathcal{J}(\varrho, \phi)$ with respect to ϱ ((27))
      Update the termination-network with respect to ϖ ((28))
    end if
  end for
  Replicate ϱ into ϱ' at every interval of κ steps.
end for

of the UAV u:

$$\tilde{U}(\psi_{o_u(t)'}, \zeta|\varrho') = (1 - \alpha(\psi_{o_u(t)'}, \zeta|\varpi))\partial^{sr}(\phi_{o_u(t)'}, \zeta|\varrho') + \alpha(\psi_{o_u(t)'}, \zeta|\hbar)\partial^{sr}(\psi_{o_u(t)'}, \zeta'|\varrho'), \quad (25)$$

where $\zeta' = \text{argmax}_{\zeta\in\Theta}\partial^{sr}(\psi_{o'_u(t)}, \zeta|\varrho') \cdot \varpi$. ∂^{sr} denotes the anticipated discounted future state occupancy of performing the option.

In Algorithm 2, we describe the training process of option-network in detail. We begin by initializing the parameters of the networks and the replay buffer. After that, we sample the interaction experience between UAVs and UAV-assisted MEC environments for model training. The loss of the option-network consists of three parts: the local observation information reconstruction loss $\mathcal{J}(\bar{\phi}, \phi)$, the loss for reward weights $\mathcal{J}(\varpi, \phi)$ and the option-network $\mathcal{J}(\varrho, \phi)$. First, we optimize the local observation information reconstruction network by minimizing the following two losses:

$$\mathcal{J}(\bar{\phi}, \phi) = \left(g_{\bar{\phi}}(\phi_{o_u(t)}) - o_u(t)\right)^2,$$
$$\mathcal{J}(\varrho, \phi) = \left(r_u(t) - \phi_{o_u(t)} \cdot \varpi\right)^2. \quad (26)$$

Second, we optimize the option network by minimizing the following standard L2 loss:

$$\mathcal{J}(\varrho, \phi) = \left(y - \partial^{sr}(\psi_{o_u(t)}\right)^2, \quad (27)$$

where $y = \phi_{o_u(t)} + \gamma\tilde{Y}(\psi_{o_u(t)}, \zeta|\varrho)$. Finally, we optimize the reward weights by minimizing the following loss:

$$\varepsilon = \varepsilon - \beta_\varepsilon \frac{\partial\alpha(\phi_{o_u(t)'}, \zeta|\varepsilon)}{\partial\varepsilon}\left(A(\phi_{o_u(t)'}, \zeta|\varrho') + \varsigma\right), \quad (28)$$

where $A(\phi_{o_u(t)'}, \zeta|\varrho')$, the advantage function, is estimated as $\partial^{sr}(\phi_{o_u(t)'}, \zeta|\varrho') \cdot \varpi - \max_{\zeta\in\Theta}\partial^{sr}(\phi_{o_u(t)'}, \zeta|\varrho') \cdot \varpi$, and ς denotes the normalization factor.

C. Partially Observable Mean Field Optimization Module

Most existing JTCTO methods [21], [22], [23], [24], [25] are not scalable to UAV-assisted MEC environments with a large number of UAVs, as their computational complexity grows exponentially with the number of UAVs. We introduce an improved mean field algorithm, the partially observable mean field optimization module, to address this challenge. Specifically, we propose a partially observable mean field optimization module into our JTCTO method, where the interactions among the UAVs in the partially observable UAV-assisted MEC environment are approximated as those between an individual UAV and the mean influence from the entire UAVs or nearby UAVs. In other words, our algorithm simplifies interaction modeling by calculating the average impact between a single UAV and all UAVs or adjacent UAVs rather than considering the complex interaction between all UAVs. This method can reduce the computational complexity and make the algorithm more suitable for large-scale UAV-assisted MEC system environments. Next, we will introduce the mean field algorithm and the corresponding improved algorithm in detail.

First, the mean-field algorithm. We factorize the Q-function solely based on the local pairwise interactions as follows:

$$Q_u(S(t), A(t)) = \frac{1}{\ell_u}\sum_{u'\in\ell(u)} Q_u(S(t), a_u(t), a_{u'}(t)), \quad (29)$$

where $\ell(u)$ denotes the set of the adjacent UAVs of UAV u, and its size, $\ell_u = |\ell(u)|$, is defined by the configuration in UAV-assisted MEC scenario. It is essential to highlight that despite significantly simplifying the complexity of interactions among UAVs, the pairwise approximation of the UAV and its nearby UAVs still retains the implicit global interactions between any pair of UAVs. Based on the mean field theory [38], the pairwise interaction $Q_u(S(t), a_u(t), a_{u'}(t))$ can be approximated. Here, we consider discretizing the JTCTO strategies of UAVs: $a_u(t) \triangleq [a_u^1(t), \ldots, a_u^\epsilon(t)]$, where ϵ denotes the number of possible actions. We compute the mean JTCTO strategy $\bar{a}_u(t)$ according to the neighborhood $\ell(u)$ of UAV u, and then define the one-hot JTCTO strategies $a_{u'}(t)$ of each neighbor u' in terms of the sum of $\bar{a}^j$ and a slight variation $\mu a_{u,u'}$ as

$$a_{u'} = \bar{a}_u + \mu a_{u,u'}, \quad \text{where } \bar{a}_u(t) = \frac{1}{\ell_u}\sum_{u'} a_{u'}(t), \quad (30)$$

where $\bar{a}_u \triangleq [\bar{a}_u^1, \ldots, \bar{a}_u^\epsilon]$ represents the empirical distribution of the JTCTO strategies performed by the neighbors of UAV u. According to Taylor's theorem, if the pairwise Q-function $Q_u(S(t), a_u(t), a_{u'}(t))$ is twice-differentiable with respect to the JTCTO strategies $a_{u'}(t)$ performed by neighbor u', it can be expanded and approximated as follows:

$$Q_u(S, A) = \frac{1}{\ell_u}\sum_{u'\in\ell(u)} Q_u(S(t), a_u(t), a_{u'}(t)) \quad (31)$$

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 08,2026 at 04:39:35 UTC from IEEE Xplore. Restrictions apply.

$$= \frac{1}{\ell_u} \sum_{u' \in \ell(u)} [Q_u(S, a_u, \bar{a}_u) + \nabla_{\bar{a}_u} Q_u(S, a_u, \bar{a}_u) \cdot \mu a_{u,u'} + \frac{1}{2} \mu a_{u,u'} \cdot \nabla^2_{\tilde{a}_{u,u'}} Q_u(S, a_u, \tilde{a}_{u,u'}) \cdot \mu a_{u,u'}]$$

$$= Q_u(S, a_u, \bar{a}_u) + \nabla_{\bar{a}_u} Q_u(S, a_u, \bar{a}_u) \cdot \left[\frac{1}{\ell_u} \sum_{u' \in \ell(u)} \mu a_{u,u'} \right] + \frac{1}{2\ell_u} \sum_{u' \in \ell(u)} [\mu a_{u,u'} \cdot \nabla^2_{\tilde{a}_{u,u'}} Q_u(S, a_u, \tilde{a}_{u,u'}) \cdot \mu a_{u,u'}]$$

$$= Q_u(S, a_u, \bar{a}_{u'}) + \frac{1}{2\ell_u} \sum_{u'} R^u_{S,a_{u'}}(a_u) \approx Q_u(S, a_u, \bar{a}_{u'}).$$

Based on (30), we can conclude that $\sum_{u'} \mu a^{u'} = 0$ in (30), meaning the first-order term can be eliminated. $R^u_{S(t),a_u(t)}(a_{u'}(t)) \triangleq \mu a_{u,u'} \cdot \nabla^2_{\tilde{a}_{u,u'}} Q_u(S(t), a_u(t), \tilde{a}^{u,u'}) \cdot \mu a_{u,u'}$ represents the residual of Taylor polynomial with respect to $\tilde{a}_{u,u'} = \bar{a}_u + \digamma_{u,u'} \mu a_{u,u'}$ and $\digamma_{u,u'} \in [0, 1]$. Moreover, this residual term possesses the following property: if the value function $Q_u(S(t), a_u(t), a_{u'}(t))$ is a continuous function with $\imath$-th order derivatives, then the value of this residual term is bounded to be $[-2\imath, 2\imath]$. Thus, $R^u_{S(t),a_u(t)}(a_{u'}(t))$ serves as a minor variation close to zero.

Second, the partially observable mean field optimization module. As shown in Fig. 2, we consider a partially observable multi-UAV-assisted MEC system environment, where UAVs communicate with other UAVs and ground users within a fixed coverage radius. Our setup follows that of [29], [30], [31], but we further relax the assumption of global state visibility. Specifically, we modify the update in (32) by keeping a categorical distribution for the mean action parameter. Instead of relying on global information, we use only the local observation $o_u(t)$ of the UAV u. The corresponding Q-update equation for our approach is presented in (32). Given that the conjugate prior to a categorical distribution is the Dirichlet distribution, we employ a Dirichlet prior for this parameter. Let L^a denote the size of the action space. We consider defining $\eth$ to denote the parameters of the Dirichlet $(\eth_1, \ldots, \eth_{L^a})$. We consider defining υ to denote the parameters of the categorical distribution $(\upsilon_1, \ldots, \upsilon_{L^a})$. We consider defining Z to denote an observed action sample $(x_1, \ldots, x_G)$ of G UAVs. Then the Dirichlet for UAV u can be represented as $\mathcal{D}_u(\upsilon|\eth) \propto \upsilon_1^{\eth_1 - 1} \ldots \upsilon_{L^a}^{\eth_{L^a} - 1}$ and the likelihood is expressed as $p(Z|\upsilon) \propto \upsilon_1^{Z=1} \ldots \upsilon_{L^a}^{Z=L^a} \propto \upsilon_1^{\daleth_1} \ldots \upsilon_{L^a}^{\daleth^a_L}$, where $[Z = i]$ denotes the Iverson bracket, which yields 1 if $Z = i$ and 0 otherwise. This value represents the frequency of each category $(\daleth_1, \ldots, \daleth_{L^a})$, represented by $\daleth$. Through Bayesian updating, the posterior is represented as a Dirichlet distribution as described in (34) where the parameters of this Dirichlet are determined by $\mathcal{D}_u(\upsilon|\eth + \daleth)$. Therefore, we update Q by the (32),

$$Q_u^{t+1}(o_u, a_u, \bar{a}_u) = (1 - \Gamma) Q_u^t(o_u(t), a_u(t), \bar{a}_u(t)) + \Gamma[r_u(t) + \gamma V_u(t)(o_u(t+1))], \quad (32)$$

$$\mathcal{D}_u(\upsilon) \propto \upsilon_1^{\eth_1 - 1 + \daleth_1} \ldots \upsilon_{L^a}^{\eth_{L^a} - 1 + \daleth_{L^a}}; \mathcal{D}_u(\upsilon|\eth + \daleth), \quad (33)$$

Algorithm 3: The PTMF-MAAC Algorithm-Based JTCTO Method.

Input: the network structure for Actor, Critic, Option, and Termination.
Output: the flight trajectory $\omega_u(t)$, the task offloading ratio $(\chi^l_{m,u}(t), \chi^n_{m,u}(t))$, the transmitting power $p_u(t)$ of the UAV u.
for each UAV agent $u = \{1, \ldots, U\}$ **do**
  Initialize the Main-Actor network, Target-Actor network, Main-Critic network and Target-Critic network
  Initialize the Option network and Termination network
end for
Initialize the experience replay pool E_u, sampling size D, discount factor γ, learning rate δ, soft update frequency ξ.
for *each episode* $epi = \{1, \ldots, N_{epi}\}$ **do**
  for *each slot* $t = \{1, \ldots, T\}$ **do**
    Get initial observation information of all UAVs $S(t) = \{o_1(t), \ldots, o_U(t)\}$.
    for *each UAV agent* $u = \{1, \ldots, U\}$ **do**
      Choose $\zeta_u(t)$ according to (25).
      Take the action $a_u(t)$ according to $\pi_u(o_u(t))$.
    end for
    Compute the new average action $\bar{A}(t) = [\bar{a}_1(t), \ldots, \bar{a}_U(t)]$.
    Execute joint actions $A(t) = [a_1(t), \ldots, a_U(t)]$ and get the reward $R(t) = [r_1(t), \ldots, r_U(t)]$ and the next state $S(t+1)$.
    Store $\langle S(t), A(t), R(t), S(t+1), \bar{A}(t), \zeta, u \rangle$ in replay buffer.
    $S(t) = S(t+1)$.
    Choose $\zeta'_u(t)$ if $\zeta_u(t)$ terminates for each UAV agent u.
    if *episode* $epi > \hat{epi}$ **then**
      Sample a minibatch of samples $\langle S(t), A(t), R(t), S(t+1), \bar{A}(t), \zeta, u \rangle$
      Minimizing the loss of critic-network with respect to θ_u (22)
      the policy transfer optimization module computes policy transfer loss $\mathcal{J}^{tr}_u$.
      Minimizing the loss of actor-network with respect to φ_u (23)
    end if
    Update the parameters of the target networks for each UAV u with learning rates τ_ϕ and τ_θ:
$$\phi^u_- \leftarrow \tau_\phi \phi^u + (1 - \tau_\phi) \phi^u_-$$
$$\theta^u_- \leftarrow \tau_\theta \theta^u + (1 - \tau_\theta) \theta^u_-$$
  end for
  Optimize the option-network (**Algorithm 2**)
end for

where Γ is the learning rate. Furthermore, the mean field value function of the UAV can be defined by the (34),

$$V_u(o_u(t+1)) = \sum_{a_u(t+1)} \pi_u(a_u(t+1)|o_u(t+1), \bar{a}_u(t)) \mathbb{E}_{\bar{a}_u(\boldsymbol{a}_{-u}(t)) \sim \boldsymbol{\pi}_{-u}(t)} [Q_u(o_u(t+1), a_u(t+1), \bar{a}_u)]. \quad (34)$$

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 08,2026 at 04:39:35 UTC from IEEE Xplore. Restrictions apply.

In each time interval, $\bar{a}_u(t)$ is derived from the strategy $\pi'_u(t)$ of the neighbor u' in the previous time interval. Additionally, the action within its strategy parameters is treated as the average action from the previous interval. The update process is as follows:

$$\bar{a}_u^i(t) \sim \mathcal{D}_u(\upsilon; \eth + \daleth); \bar{a}_u(t) = \frac{1}{K}\sum_{i=1}^{i=K} \bar{a}_u^i(t), \tag{35}$$

The average neighbor action $\bar{a}_u(t)$ is computed using (35), and the updated strategy is then obtained by applying the Boltzmann distribution, as expressed in the following form:

$$\pi_u(a_u|o_u, \bar{a}_u(t')) = \frac{\exp\left(\wp Q_u(o_u, a_u, \bar{a}_u(t'))\right)}{\sum_{a_{u'}\in\mathcal{A}_u} \exp\left(\wp Q_u(o_u, a_{u'}, \bar{a}_u(t'))\right)}, \tag{36}$$

where t' represents $t-1$. $\wp$ is a temperature parameter. Through the continuous iterative updates of (35) and (36), all the UAV strategies can be enhanced, enabling the proposed algorithm to achieve a significant cumulative return. To enhance scalability and adaptability, we replace the mean field aggregation method used in [29], [30], [31] with a Bayesian update approach based on the Dirichlet distribution, as described in (33). In (35), we draw K samples from this distribution to estimate the partially observable mean action ($\bar{a}$), thereby relaxing the constraint of full global state observability.

D. The Whole Training Process of the PTMF-MAAC Algorithm

Algorithm 3 shows that the whole training process of PTMF-MAAC-based JTCTO algorithm. The PTMF-MAAC-based JTCTO algorithm is implemented based on the multi-agent actor-critic framework. As aforementioned, there are U PTMF-MAAC agents in the UAV-assisted MEC system, each running on a UAV. By interacting with a local UAV, each UAV agent makes independent JTCTO decisions taken in that UAV. Specifically, Step 1∼7 represents the initialization of PTMF-MAAC-based JTCTO algorithm training. Step 8∼18 denotes the gathering of training data. Step 21 denotes the update of the critic-network. Step 27∼33 denotes the training of the policy transfer optimization module. Step 24 denotes the update of actor-network. Step 26 denotes the update of the target-network. Step 28 denotes the training process of the option-network.

E. The Complexity Analysis of PTMF-MAAC Algorithm

According to Fig. 2, the PTMF-MAAC algorithm mainly includes the partially observable mean field optimization module and the policy transfer optimization module. Therefore, we mainly analyze the complexity of these two modules. Specifically, first, we provide an analysis of the time complexity of the partially observable mean field optimization module in each training iteration. U represents the number of UAV agents. L^a represents the size of the UAV agent's action space. K represents the number of samples taken from the Dirichlet distribution, i.e., (35). In the proposed partially observable UAV-assisted MEC system environment, we consider that UAVs only use other UAVs and ground user information within their communication range as observation information for model training. Therefore, the number of other UAVs within the observation range of each UAV is constant (or has an upper limit). In other words, the complexity of observing neighboring UAVs can be considered as $\mathcal{O}(1)$ or independent of U. In one update (i.e., one-time step), each UAV agent mainly performs four operations: (a) For each state action combination $(o_u(t), a_u(t), \bar{a}_u(t))$, update the Q value according to (32). The time complexity here is usually $\mathcal{O}(1)$. (b) Bayesian update of Dirichlet distribution (i.e., (32)). For UAV agent u, it is necessary to update the Dirichlet parameters $\eth + \daleth$ based on the observed action statistics $(\daleth_1, \ldots, \daleth_{L^a})$ of other UAVs within the communication range. This part of the update usually only requires adding L parameters, with a complexity of about $\mathcal{O}(L^a)$. (c) Sampling and calculating mean actions from Dirichlet distribution, i.e., (33). Each time a set of $(\upsilon_1, \ldots, \upsilon_{L^a})$ is sampled from Dirichlet $(\eth + \daleth)$, it needs to be executed K times. In the proposed system model, the parameter $\mathcal{O}(L^a)$is relatively small. Therefore, the complexity of the sampling algorithm can be regarded as $\mathcal{O}(L^a)$, and the total complexity is $\mathcal{O}(K \times L^a)$. In addition, the algorithm complexity of averaging the effects of K samples is still $\mathcal{O}(K \times L^a)$. (d) Boltzmann policy update ((36)). The action space size is L^a, and computing a single softmax operation requires performing exponentiation and normalization on L Q-values, resulting in a complexity of $\mathcal{O}(K)$. In summary, for a single UAV, the main complexity of one update is concentrated in Dirichlet sampling ($\mathcal{O}(K \times L^a)$) and Boltzmann strategy update ($\mathcal{O}(L^a)$), with an overall complexity of $\mathcal{O}(K \times L^a)$+$\mathcal{O}(L) \approx \mathcal{O}(K \times L^a)$. Since there are a total of U UAVs, and assuming that the number of neighbors of each UAV is a constant $U', (U' \ll U)$ in some observable environment settings, the total complexity of a global update is about $\mathcal{O}(U \times K \times L^a)$.

Second, we provide an analysis of the time complexity of the policy transfer optimization module. The policy transfer optimization module mainly contains four key steps. (a) Policy selection and execution (sampling phase). Each UAV agent selects an option and action to execute actions and store data. The time complexity of this part is $\mathcal{O}(U \times T)$. T is the length of the episode. (b) The update of critic-network. To calculate the loss of critic-network, it is necessary to calculate the future discount return, involving the calculation of t time steps. The complexity of this part is $\mathcal{O}(U \times T)$. (c) The update of actor-network. Calculate the loss of actor-network, including policy gradient calculation, Kullback-Leibler divergence regularization, and policy transfer loss. The complexity of this part is $\mathcal{O}(U \times T)$. (d) The option module updates complexity. Each update step involves: the complexity of sampling a batch of size B from the replay buffer is $\mathcal{O}(B)$, the complexity of optimizing state reconstruction and reward weights is $\mathcal{O}(B \times d)$, the complexity of optimizing option network is $\mathcal{O}(B \times d)$, the complexity of updating termination probabilities is $\mathcal{O}(B)$. B indicates the batch size. d represents the size of the neural network. In summary, the complexity of sampling part is $\mathcal{O}(U \times T)$, the complexity of critic-network training is $\mathcal{O}(U \times T)$, the complexity of actor-network training is $\mathcal{O}(U \times T)$, and the complexity of option-network training is $\mathcal{O}(B)$. Therefore, the time complexity of this part is $\mathcal{O}(U \times T)$. Through the analysis of the policy transfer optimization module and the partially observable mean field

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 08,2026 at 04:39:35 UTC from IEEE Xplore. Restrictions apply.

optimization module, we can conclude that the time complexity of the PTMF-MAAC algorithm is $\mathcal{O}(U \times K \times L^a + U \times T)$.

F. The Optimality Analysis of PTMF-MAAC Algorithm

Based on Section V-C, we further analyze the optimality of the PTMF-MAAC algorithm, which mainly includes the following four aspects: first, the algorithm uses the classical Q-learning update formula, satisfies the Robbins Monro condition, and uses the contraction of the Bellman operator to ensure that the Q-value can converge to a fixed point, thus providing a theoretical basis for the optimal strategy. Second, PTMF-MAAC accurately estimates mean action. The local mean action is updated by the Bayesian method using Dirichlet prior, and the estimation is obtained by sampling. This method can make the mean action estimation unbiased and approximate to the global mean when there are enough samples so as to provide accurate mean field information for subsequent Q-value updates. Third, PTMF-MAAC uses the Boltzmann soft strategy, which makes the strategy greedy with the convergence of Q-value, and finally, it converges to a stable strategy combination. This combination satisfies the fact that each UAV agent cannot unilaterally improve the return under local observation; that is, it constitutes a local Nash equilibrium, which represents a stable best point. Fourth, PTMF-MAAC has a good balance between exploration and global optimality. The noise introduced by Dirichlet sampling increases the exploration ability and helps agents jump out of the local optimal trap. With a reasonable number of samples and training time, the additional exploration brought by sampling can help to approach the global optimal Nash equilibrium. Although some observable constraints may lead to some deviation, the overall performance is expected to be improved. Finally, PTMF-MAAC solves the local optimal problem of the traditional mean field algorithm [29], [30], [31] by introducing the Dirichlet sampling mechanism, which can help the UAV agent better avoid falling into the local optimal solution, thus improving the possibility of convergence to the global optimal Nash equilibrium. In theory, the update rule conforms to the condition of the stochastic approximation algorithm, which ensures that it converges to a stable equilibrium point, which satisfies the Nash equilibrium condition. After enough training rounds, the proposed algorithm can effectively achieve Nash equilibrium, and its convergence performance is better than the traditional mean field algorithm [29], [30], [31].

VI. Simulation Experiment

A. Simulation Environment Description

In this section, numerical experiments are carried out to assess the effectiveness of the PTMF-MAAC method. Here, we consider a UAV-assisted MEC scenario with multiple UAVs and ESs. In our system environment, we set the number of UAVs to 4∼64, the number of SDs to 50∼150, and the number of ESs to 2∼8. Similar to this work [35], we consider all UAVs flying in an area of 3000×3000 meters and SDs randomly distributed in this area. Source code is provided on https://github.com/gaozhen610351/PTMF-MAAC/issues/1. Table III describes the main simulation parameter configurations.

TABLE III
Simulation Experiment Parameter Setting

Parameters	Value
$d_m(t)$	[0.5, 6] Mbits [22], [44]
$\rho_m(t)$	[50, 250] cycles/bit [22], [44]
$\lambda_m(t)$	0.4∼1.2 task/sec [22], [44]
N	2∼8
M	50∼150 [25], [44]
U	4∼64 [22]
$\jmath_{\max}$	30 m [25], [44]
H	40∼120 m [25], [44]
$F_u, u \in \mathcal{U}$	3.6∼6.6 GHz [44], [45]
$F_n, n \in \mathcal{N}$	6.6∼11.6 GHz [44], [45]
$P_{\max}$	5 W [44], [45]
$p_m, m \in \mathcal{M}$	0.1 W [44], [45]
$p_u^a, u \in \mathcal{U}$	0.1∼5 W
$\sigma_u^2, u \in \mathcal{U}$	-100 dBm [46]
$\sigma_n^2, n \in \mathcal{N}$	-100 dBm [46]
$W_u, u \in \mathcal{U}$	5∼10 MHz [22]
ϑ	10^{-28}
φ	$\pi/4$

B. Training Setup

Based on the description in Section V, the PTMF-MAAC algorithm mainly contains the policy transfer optimization module and the partially observable mean field optimization module. The PTMF-MAAC algorithm is implemented based on the multi-agent actor-critic framework. Regarding the training parameters of these two modules, we set them as follows. Specifically, the actor network comprises two fully-connected hidden layers. The critic network consists of two fully connected hidden layers. The Adam optimizer is utilized with a learning rate of 0.001 for the critic network and 0.0001 for the actor-network, while $\tau = 0.01$ is applied for updating the target networks. We set the discounted factor γ to 0.95 and the mini-batch size to 200. We maintain a replay buffer size of 10^6.

C. Method Comparison

To evaluate the effectiveness of PTMF-MAAC, we use the following benchmarks. First, to verify the effectiveness of the policy transfer optimization module (PT), we compare the JTCTO algorithm without policy transfer, i.e., AGIN-MADDPG [39]. Moreover, we compare the existing JTCTO method based on the transfer algorithm, i.e., KMASAC [40]. Second, to verify that the PTMF-MAAC algorithm adapts to large-scale UAV-assisted MEC systems, we compare it with the HT3O algorithm [28]. Finally, we compare the existing JTCTO method, namely MAPPO-COT [22]. To verify the effectiveness of the proposed partially observable mean-field module, we compare PTMF-MAAC with the JTCTO algorithm based on mean-field theory, similar to these methods [29], [30], [31], namely RAT. In addition, we compare PTMF-MAAC with the trajectory control and task offloading (TCTO) algorithm based on graph RL, similar to these methods [41], [42], namely GRL-TCTO.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 08,2026 at 04:39:35 UTC from IEEE Xplore. Restrictions apply.

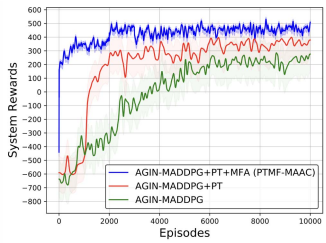

Fig. 4. Convergence and system rewards of PTMF-MAAC employing various optimization components.

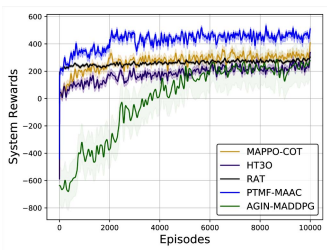

Fig. 5. Convergence and system rewards of various algorithms versus episodes.

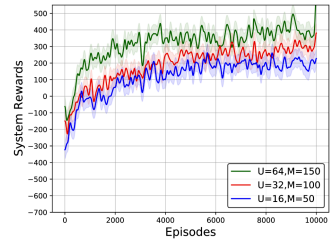

Fig. 6. Performance evaluation of PTMF-MAAC using varying quantities of UAVs.

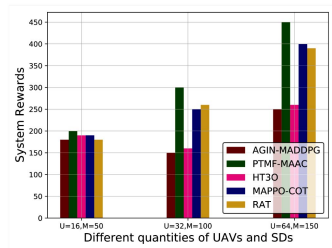

Fig. 7. System rewards of the algorithm under different quantities of UAVs and SDs.

D. Simulation Experiment Analysis

As illustrated in Fig. 4, we demonstrate the convergence and system rewards of PTMF-MAAC utilizing different optimization components. We take AGIN-MADDPG [39] as the baseline and integrate the policy transfer optimization module and the partially observable mean field optimization module into this algorithm in sequence. In this experiment, the policy transfer optimization module and the partially observable mean field optimization module are called PT and MFA, respectively. Based on the AGIN-MADDPG, we first introduce a PT module. As illustrated in Fig. 4, compared with the AGIN-MADDPG algorithm, the AGIN-MADDPG+PT algorithm can generate higher system rewards and accelerate the algorithm's convergence. For example, compared with the AGIN-MADDPG algorithm, the convergence speed of the AGIN-MADDPG+PT algorithm is accelerated by 20.23%, and the reward is increased by 65.12%. This is because the policy transfer optimization module is proposed to determine which UAV's JTCTO strategy is helpful for each UAV and when to terminate the strategy to accelerate the learning efficiency of the UAV. Second, based on the AGIN-MADDPG+PT, we further introduce an MFA module. As illustrated in Fig. 4, compared with the AGIN-MADDPG+PT algorithm, the AGIN-MADDPG+ISM+MFA (PTMF-MAAC) algorithm can generate higher system rewards. For example, compared with the AGIN-MADDPG+PT algorithm, the reward of the PTMF-MAAC algorithm increased by 28.57%, and the convergence speed of the PTMF-MAAC algorithm has improved by 50.23%. This is because we consider that for a certain UAV, the effect of all other UAVs on it can be replaced by a mean value, which significantly reduces the model space and thus improves the reward and convergence speed of the algorithm.

As illustrated in Fig. 5, we demonstrate various algorithms' convergence and system rewards. In Fig. 5, we find that compared to other algorithms, the PTMF-MAAC algorithm always maintains a higher reward. For example, compared with different algorithms, the reward of the PTMF-MAAC algorithm is increased by 55.25%∼70.33%. This is because, first, we minimize energy consumption and latency by jointly optimizing UAV flight trajectories, task offloading policies, and resource allocation policies, and ultimately, UAVs can provide computational services to ground users at optimal locations. Second, we propose a policy transfer optimization module, which allows distributed UAVs to quickly and economically improve their learning performance by sharing their learned intelligence. Finally, we argue that for a given UAV, the effect of all other UAVs on it can be replaced by an average value, which significantly reduces the model space and thus improves the reward of the algorithm.

As illustrated in Fig. 6, we assess the performance of PTMF-MAAC using varying quantities of UAVs. In Fig. 6, it can be observed that the model converges at a slower rate with an increase in the number of UAVs. For example, when the number of UAVs is 16, the algorithm converges at 3800 episodes. When the number of UAVs is 32, the algorithm converges with 4500 episodes. When the number of UAVs is 64, the algorithm converges into 7800 episodes. This occurs because, with a substantial increase in UAVs, learning becomes challenging due to the curse of dimensionality and the exponential rise in UAV interactions. Further, this suggests that the convergence performance of PTMF-MAAC is inversely proportional to the number of UAVs. This could be attributed to the higher number of UAVs, which is leading to increased challenges in obtaining the desired UAV locations. It is worth noting that our proposed algorithm still converges when the number of UAVs is 64. This is because we consider that for a certain UAV, the effect of all other UAVs on it can be replaced by a mean value, which significantly reduces the model space and thus adapts to large-scale scenarios.

As shown in Fig. 7, we investigate the impact of different quantities of UAVs and SDs on the algorithm's system rewards.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 08,2026 at 04:39:35 UTC from IEEE Xplore. Restrictions apply.

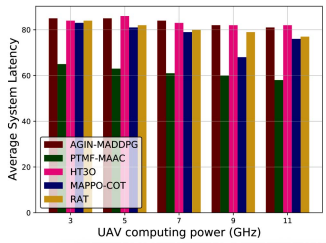

Fig. 8. Average system latency of UAVs with different computing power.

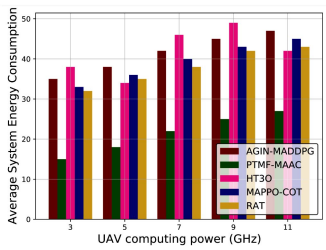

Fig. 9. Average system energy consumption of UAVs with different computing power.

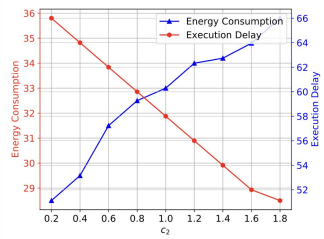

Fig. 10. Energy consumption and latency change with the weight coefficient c_2.

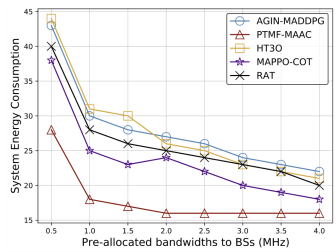

Fig. 11. System energy consumption at different bandwidths pre-allocated to ESs.

From Fig. 7, we can observe that the rewards for all algorithms also increase as the number of UAVs and SDs increases. This is because, with more UAVs and SDs, more offloading tasks are generated and completed, leading to higher rewards for the algorithms. Notably, compared to other algorithms, ours consistently maintains the highest rewards. This is because our algorithm incorporates a partially observable mean field optimization module, which reduces the interaction complexity between each UAV and the environment, allowing the algorithm to adapt to large-scale UAV-assisted MEC system environments.

As shown in Fig. 8, we explore the average system latency of UAVs with different computing power. In Fig. 8, we can find that the latency of all algorithms gradually decreases as the computational power of the UAV increases. This is because as the computational power of the UAV increases, the offloaded tasks will be allocated more computational resources for processing, thus leading to a low computational latency. It is worth noting that the PTMF-MAAC algorithm has always maintained low latency. For example, compared with other algorithms, the PTMF-MAAC algorithm's latency is reduced by 14.29%$\sim$26.83%. This is due to the proposed policy transfer algorithm, which determines which UAV's JTCTO strategy is helpful for each UAV and when to terminate it. It enhances the collaboration between UAVs to handle offloaded tasks and reduces latency.

As shown in Fig. 9, we explore UAVs' average system energy consumption with different computing power. In Fig. 9, we find that as UAVs' computing power increases, all algorithms' energy consumption gradually increases. This is because as the computational power of UAVs increases, more offloaded tasks are chosen to be processed on UAVs, further increasing the energy consumption of UAVs. It is worth noting that the PTMF-MAAC algorithm has always maintained low energy consumption. For example, compared with other algorithms, the energy consumption of the PTMF-MAAC algorithm is reduced by 47.62%$\sim$54.17%. This is because the proposed policy transfer algorithm and mean-field approximation module can increase the collaboration between multiple UAVs through information sharing and simplify the UAV interaction process, thereby reducing the energy consumption of task processing.

As demonstrated in Fig. 10, we explore the energy consumption and latency change with the weight coefficient c_2. In this experiment, we consider setting c_2 from 0.2 to 1.8 and setting c_1 to 1. In Fig. 10, we find that the smaller c_2 is, the more significant the proportion of energy consumption. As the weight c_2 increases, the execution delay becomes more visible, and more offloaded tasks are processed by the UAV, reducing the delay and increasing energy consumption. When c_2 reaches a specific size, the reduction in execution delay ceases as the UAVs' computing capacity is constrained, and increased tasks result in elevated processing delays. From the above experiment, c_2 is already a fixed value, while the value range of c_2 is between 0.2 and 1.8. Through experiments, it can be observed that different values of c_2 have an impact on energy consumption and execution delay. The experiment mainly analyzed the energy consumption and latency trends under different values of c_2, but we did not directly provide an optimal weight combination. Therefore, the optimal c_2 needs to be determined based on specific optimization objectives. Specifically, first, to minimize the energy consumption of the UAV-assisted MEC system, a smaller c_2 (e.g., 0.2$\sim$0.6) can be selected, resulting in lower energy consumption at the cost of higher execution delay. Second, to reduce execution delay, a larger c_2 (e.g., 1.2$\sim$1.6) can be selected, leading to lower execution delay but higher energy consumption. Finally, to achieve a balance between energy consumption and execution delay, observing the experimental trends suggests that $c_2 \approx 1$ may be a suitable trade-off point, where both energy consumption and execution delay remain relatively balanced without extreme cases.

As demonstrated in Fig. 11, we explore the energy consumption change with different bandwidths pre-allocated to ESs.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 08,2026 at 04:39:35 UTC from IEEE Xplore. Restrictions apply.

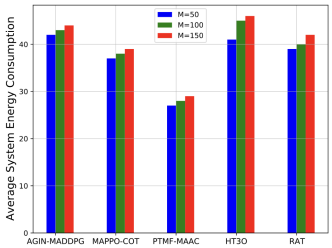

Fig. 12. The average system energy consumption of different methods with different number of SDs.

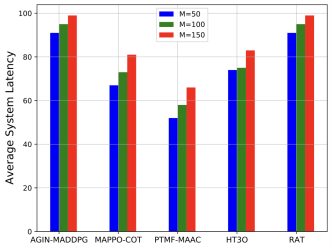

Fig. 13. The average system latency of different methods with different number of SDs.

As the bandwidth allocated to the ES increases, SDs will be allocated more bandwidth from the ESs when offloaded tasks are determined to be processed on the ES so that SDs will get faster transfer rates and, ultimately, energy consumption is reduced. It is worth noting that the PTMF-MAAC algorithm maintains low energy consumption. For example, the PTMF-MAAC algorithm reduces energy consumption by 22.73%∼34.62%.

As shown in Fig. 12, we explore different methods' average system energy consumption with varying numbers of SDs. In Fig. 12, we find that as the number of SDs increases, the energy consumption of all algorithms continues to grow. This is because as the number of SDs increases, more offloaded tasks will be processed, thus increasing energy consumption. It is worth noting that the PTMF-MAAC algorithm has always maintained low energy consumption. For example, compared with other algorithms, the energy consumption of the PTMF-MAAC algorithm is reduced by 24.32%∼33.33%. This is because our algorithm is based on the implementation of the multi-agent actor-critic framework and incorporates a policy transfer algorithm and a mean-field approximation module that can increase collaboration among multiple UAVs by sharing information and simplifying the UAV interaction process, thus reducing the energy consumption of task processing.

As demonstrated in Fig. 13, we explore different methods' average system latency with different SDs. In Fig. 13, we find that as the number of SDs increases, the latency of all algorithms continues to grow. This is because as the number of SDs increases, more offloaded tasks will be processed, thus increasing latency. It is worth noting that the PTMF-MAAC algorithm has always maintained low latency. For example, compared with other algorithms, the latency of the PTMF-MAAC algorithm is reduced by 19.12%∼38.89%. This is because our algorithm is based on the implementation of the multi-agent actor-critic framework and incorporates a policy transfer algorithm and a mean-field approximation module that can increase collaboration among multiple UAVs by sharing information and simplifying the UAV interaction process, thus reducing the latency of task processing.

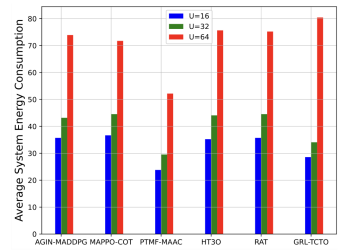

Fig. 14. The average system energy consumption of different methods with different number of UAVs.

As shown in Fig. 14, we explored the average energy consumption of different methods under varying numbers of UAVs. We compare the proposed algorithm with the trajectory control and task offloading (TCTO) algorithm based on graph RL, similar to these methods [41], [42], namely GRL-TCTO. In Fig. 14, we find that as the number of UAVs increases, the average energy consumption of all algorithms continues to grow. This is because as the number of UAVs increases, more offloaded tasks will be processed, thus increasing energy consumption. It is worth noting that the PTMF-MAAC algorithm has always maintained low energy consumption. For example, compared with other algorithms, the energy consumption of the PTMF-MAAC algorithm is reduced by 19.09%∼34.62%. This is because our algorithm integrates partially observable mean field approximation modules, which reduces the complexity of large-scale UAV interactions and improves the collaboration between UAVs, thereby obtaining optimal decisions and reducing the energy consumption of task execution. It is worth noting that when the number of UAVs is 16 and 32, GRL-TCTO exhibits lower average system energy consumption than other algorithms except for PTMF-MAAC. For example, the average system energy consumption has decreased by 22.68%∼23.47%. This is because the GRL-TCTO algorithm models the UAV-assisted MEC system environment as a graph structure, perceiving global system information through message passing between nodes, thereby obtaining the best JTCTO decisions and reducing average system energy consumption. However, when the number of UAVs is 64, GRL-TCTO achieves a higher average system energy consumption. This method relies on sharing all UAV information and manually predefining UAV communication architectures. When dealing with large-scale UAVs, identifying valuable information from globally shared information becomes difficult; the benefits of information sharing are limited and may even hinder collaborative learning, resulting in higher energy consumption. Moreover, comprehensive communication between UAVs requires high bandwidth and increases latency and computational complexity. This also verifies the necessity of our algorithm to reduce the complexity of large-scale UAV interactions by using a partially observable mean field optimization module.

As demonstrated in Fig. 15, we explored the average system latency of different methods with different numbers of UAVs. In Fig. 15, we find that the latency of all algorithms decreases

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 08,2026 at 04:39:35 UTC from IEEE Xplore. Restrictions apply.

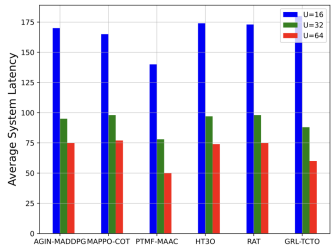

Fig. 15. The average system latency of different methods with different number of UAVs.

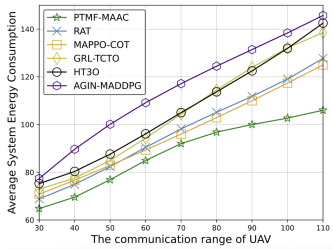

Fig. 16. The average system energy consumption of different methods with various UAV communication range.

as the number of UAVs increases. This is because as the number of UAVs increases, the offloaded tasks will be processed by more computational resources, and therefore, the computational latency will decrease. It is worth noting that the PTMF-MAAC algorithm has always maintained low latency. For example, compared with other algorithms, the latency of the PTMF-MAAC algorithm is reduced by 31.51%∼35.06%. Combining a policy transfer algorithm and a mean-field approximation module can increase collaboration among multiple UAVs and balance the computational load between UAVs by sharing information and simplifying the UAV interaction, reducing task processing delay. It is worth noting that when the number of UAVs is small, GRL-TCTO achieves lower average system latency than other algorithms except for the PTMF-MAAC algorithm. This is because GRL-TCTO models the UAV-assisted MEC system as a graph structure, perceiving global system information through message passing between nodes to obtain optimal decisions and reduce average system latency. However, GRL-TCTO also has limitations in relying on message passing between all UAVs and manually pre-defined UAV communication architectures. When the number of UAVs increases to 64, it is difficult to identify valuable information for UAV collaborative decision-making from globally shared information. This limits the benefits of UAV communication, hinders collaborative learning, and leads to the inability to obtain optimal decisions and increased latency.

As shown in Fig. 16, we investigated the impact of different UAV communication ranges on the average system energy consumption of the algorithm. In Fig. 16, we can see that as UAVs' communication range increases, all algorithms' energy consumption also increases. This is because there is a proportional relationship between the communication range of UAVs and their altitude, i.e., $\jmath_h = H \cdot \tan(\varphi)$, $\jmath_h$ is UAV communication range, H is the height of UAV, φ is the azimuth angle. Therefore, as the communication range of UAVs increases, their

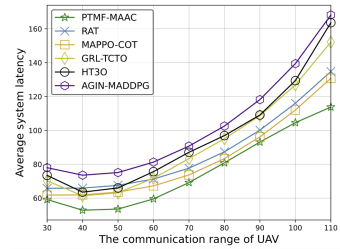

Fig. 17. The average system latency of different methods under different UAV communication ranges.

flight altitude also increases, resulting in an increase in UAV flight energy consumption and upload energy consumption for unloading tasks, ultimately leading to an increase in system energy consumption. It is worth noting that PTMF-MAAC has consistently maintained low energy consumption. For example, compared to other algorithms, PTMF-MAAC reduces energy consumption by 5.26%∼23.73%. This is because the PTMF-MAAC algorithm integrates mean field theory and policy transfer module, significantly reducing the complexity of large-scale UAV interaction and improving the algorithm's decision-making performance.

As shown in Fig. 17, we investigated the UAV communication range's impact on the algorithm's average system delay. From Fig. 17, we observe that as the UAV communication range (i.e., the height of the UAV) increases, the average system delay initially decreases and then increases. Although the expansion of UAV communication range means that UAVs can collect and process more offloading tasks, there are still enough computing resources in the system environment, thus reducing the average task execution delay. However, as the communication range continues to increase and UAVs handle an even greater number of tasks, the system's available resources become increasingly limited. This lack of computing resources will lead to higher task execution delays and eventually increase the average system delay. Notably, the PTMF-MAAC algorithm consistently maintains a lower average system latency. For instance, compared to other algorithms, PTMF-MAAC reduces the average latency by 11.54% to 30.31%. This improvement is attributed to integrating a partially observable mean-field approximation module and policy transfer mechanism, which enhance collaboration among UAVs even under partial observability. As a result, the algorithm achieves optimized decision-making and effectively minimizes average system delay.

As shown in Fig. 18, we studied the impact of different numbers of UAVs on the average system energy consumption of various algorithms. From Fig. 18, we can find that with the increase in the number of UAVs, the system energy consumption of all algorithms increases. This is because, with the rise in the number of UAVs, many offloading tasks are processed, increasing system energy consumption. It is worth noting that our algorithm has always maintained a low average system energy consumption. For example, our algorithm reduces 22.51%. In addition, as shown in Fig. 19, we also study the impact of different numbers of UAVs on the system latency of various algorithms. From Fig. 19, we can see that as the

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 08,2026 at 04:39:35 UTC from IEEE Xplore. Restrictions apply.

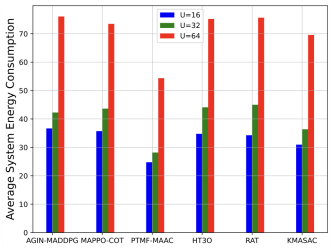

Fig. 18. The average system energy consumption of different methods with different number of UAVs.

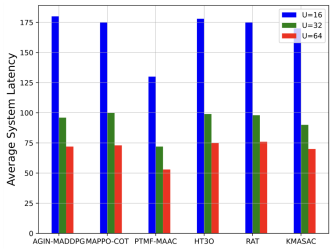

Fig. 19. The average system latency of different methods with different number of UAVs.

number of UAVs increases, the task execution latency of all algorithms will decrease. This is because as the number of UAVs increases, it is equivalent to raising the computing resources in the system environment, thereby reducing the system latency of the algorithm. It is worth noting that our algorithm has always maintained a reduced system latency. For example, compared with other algorithms, the latency of our algorithm is reduced by 25.22%∼28.12%. This is because a policy transfer algorithm is proposed to determine which UAV's JTCTO strategy is useful for each UAV and when to terminate the strategy to improve the learning efficiency of the UAV to reduce the system delay and energy consumption.

VII. Conclusion

This paper proposes a decentralized JTCTO method based on the PTMF-MAAC algorithm. Specifically, we first propose a large-scale Multi-UAV and Multi-ES UAV-assisted MEC system. Second, to improve the training efficiency of the model, we propose a new JTCTO policy transfer algorithm. Third, we propose an enhanced partially observable mean field optimization algorithm to adapt the proposed model to the large-scale UAV-assisted MEC system environment. Compared to other approaches, the PTMF-MAAC algorithm significantly reduces system latency and energy consumption, accelerates model convergence, and performs well in large-scale UAV-assisted MEC systems.

By exploring recent studies on UAV flight in MEC environments, we found that wind speed, precipitation, visibility, terrain, and signal interference may affect UAV flight. However, our research focuses on developing a more effective algorithm for the JTCTO problem. Specifically, we aim to address two key challenges in partially observable UAV-assisted MEC: the poor model scalability caused by many UAVs and the sample inefficiency issue during the model training process. In addition, we also explore a large number of recent JTCTO algorithms, e.g., [39], [41], [45], [46]. In these JTCTO algorithms, the authors usually simplify the modeling of UAV flights and do not consider the effect of environmental factors (e.g., wind speed, precipitation, visibility, terrain, and signal interference) on the UAV flight. Similar to these JTCTO methods [39], [41], [45], [46], we also simplify the modeling of UAV flight, i.e., the impact of environmental factors on UAV operation is not considered. In future work, we will explore more realistic UAV flight modeling, including wind speed, signal interference, precipitation, and visibility.

References

[1] L. Jin, M. Tang, M. Zhang, and H. Wang, "Fractional deep reinforcement learning for age-minimal mobile edge computing," in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 12947–12955.

[2] Y. Chen, J. Zhao, J. Hu, S. Wan, and J. Huang, "Distributed task offloading and resource purchasing in NOMA-enabled mobile edge computing: Hierarchical game theoretical approaches," *ACM Trans. Embedded Comput. Syst.*, vol. 23, no. 1, pp. 1–28, 2024.

[3] F. Zhang, G. Han, L. Liu, Y. Zhang, Y. Peng, and C. Li, "Cooperative partial task offloading and resource allocation for IIoT based on decentralized multi-agent deep reinforcement learning," *IEEE Internet Things J.*, vol. 11, no. 3, pp. 5526–5544, Feb. 2024.

[4] Y. Hu, M. Chen, Y. Wang, Z. Li, M. Pei, and Y. Cang, "Discrete-time joint scheduling of uploading and computation for deterministic MEC systems allowing for task interruptions and insertions," *IEEE Wireless Commun. Lett.*, vol. 12, no. 1, pp. 21–25, Jan. 2023.

[5] S. Zhou, Y. Cheng, X. Lei, Q. Peng, J. Wang, and S. Li, "Resource allocation in UAV-assisted networks: A clustering-aided reinforcement learning approach," *IEEE Trans. Veh. Technol*, vol. 71, no. 11, pp. 12088–12103, Nov. 2022.

[6] W. U. Khan, A. Ihsan, T. N. Nguyen, Z. Ali, and M. A. Javed, "NOMA-enabled backscatter communications for green transportation in automotive-Industry 5.0," *IEEE Trans. Ind. Inform.*, vol. 18, no. 11, pp. 7862–7874, Nov. 2022.

[7] Z. Liu, J. Qi, Y. Shen, K. Ma, and X. Guan, "Maximizing energy efficiency in UAV-assisted NOMA-MEC networks," *IEEE Internet Things J.*, vol. 10, no. 24, pp. 22208–22222, Dec. 2023.

[8] N. Lin, H. Tang, L. Zhao, S. Wan, A. Hawbani, and M. Guizani, "A PDDQNLP algorithm for energy efficient computation offloading in UAV-assisted MEC," *IEEE Trans. Wireless Commun.*, vol. 22, no. 12, pp. 8876–8890, Dec. 2023.

[9] H. Kang, X. Chang, J. Mišić, V. B. Mišić, J. Fan, and Y. Liu, "Cooperative UAV resource allocation and task offloading in hierarchical aerial computing systems: A MAPPO based approach," *IEEE Internet Things J.*, vol. 10, no. 12, pp. 10497–10509, Jun. 2023.

[10] M. Asim, M. ELAffendi, and A. A. Abd El-Latif, "Multi-IRS and multi-UAV-assisted MEC system for 5G/6G networks: Efficient joint trajectory optimization and passive beamforming framework," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 4, pp. 4553–4564, Apr. 2023.

[11] R. Zhou et al., "User preference oriented service caching and task offloading for UAV-assisted MEC networks," *IEEE Trans. Serv. Comput.*, vol. 18, no. 2, pp. 1097–1109, Mar./Apr. 2025.

[12] S. Akter, D. V. A. Duong, and S. Yoon, "Joint optimization of AAV trajectory, task offloading, and resource allocation in AAV-aided emergency response operations," *IEEE Internet Things J.*, vol. 12, no. 12, pp. 21944–21959, Jun. 2025.

[13] H. Xiao, X. Hu, W. Wang, Z. Su, K.-K. Wong, and K. Yang, "STAR-RIS and UAV combination in MEC networks: Simultaneous task offloading and communications," *IEEE Trans. Commun.*, early access, Jan. 29, 2025, doi: 10.1109/TCOMM.2025.3535895.

[14] T. Zeng, X. Zhang, J. Duan, C. Yu, C. Wu, and X. Chen, "An offline-transfer-online framework for cloud-edge collaborative distributed reinforcement learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 35, no. 5, pp. 720–731, May 2024.

[15] F. Fu et al., "Age of information minimization for UAV-assisted Internet of Things networks: A safe actor-critic with policy distillation approach," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 1, pp. 1265–1276, Jan./Feb. 2024.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 08,2026 at 04:39:35 UTC from IEEE Xplore. Restrictions apply.

[16] Z. Chen, J. Zhang, X. Zheng, G. Min, J. Li, and C. Rong, "Profit-aware cooperative offloading in UAV-enabled MEC systems using lightweight deep reinforcement learning," *IEEE Internet Things J.*, vol. 11, no. 12, pp. 21325–21336, Jun. 2024.

[17] W. Wang, Y. Zhang, Q. Liu, T. Wang, and W. Jia, "Edge-intelligence-based computation offloading technology for distributed Internet of Unmanned Aerial Vehicles," *IEEE Internet Things J.*, vol. 11, no. 12, pp. 20948–20957, Jun. 2024.

[18] J. Zhang, Y. Wu, G. Min, and K. Li, "Neural network-based game theory for scalable offloading in vehicular edge computing: A transfer learning approach," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 7, pp. 7431–7444, Jul. 2024.

[19] K. Zhang, D. Si, W. Wang, J. Cao, and Y. Zhang, "Transfer learning for distributed intelligence in aerial edge networks," *IEEE Wireless Commun.*, vol. 28, no. 5, pp. 74–81, Oct. 2021.

[20] X. Wang, N. Cheng, L. Ma, R. Sun, R. Chai, and N. Lu, "Digital twin-assisted knowledge distillation framework for heterogeneous federated learning," *China Commun.*, vol. 20, no. 2, pp. 61–78, 2023.

[21] C. Dai, K. Zhu, and E. Hossain, "Multi-agent deep reinforcement learning for joint decoupled user association and trajectory design in full-duplex multi-UAV networks," *IEEE Trans. Mobile Comput.*, vol. 22, no. 10, pp. 6056–6070, Oct. 2023.

[22] B. Li, R. Yang, L. Liu, J. Wang, N. Zhang, and M. Dong, "Robust computation offloading and trajectory optimization for multi-UAV-assisted MEC: A multi-agent DRL approach," *IEEE Internet Things J.*, vol. 11, no. 3, pp. 4775–4786, Feb. 2024.

[23] L. Wang, K. Wang, C. Pan, W. Xu, N. Aslam, and A. Nallanathan, "Deep reinforcement learning based dynamic trajectory control for UAV-assisted mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 21, no. 10, pp. 3536–3550, Oct. 2022.

[24] Z. Chang, H. Deng, L. You, G. Min, S. Garg, and G. Kaddoum, "Trajectory design and resource allocation for multi-UAV networks: Deep reinforcement learning approaches," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 5, pp. 2940–2951, Sep./Oct. 2023.

[25] F. Song et al., "Evolutionary multi-objective reinforcement learning based trajectory control and task offloading in UAV-assisted mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 12, pp. 7387–7405, Dec. 2023.

[26] Z. Duan et al., "Is Nash equilibrium approximator learnable?," in *Proc. Int. Conf. Auton. Agents Multiagent Syst.*, 2023, pp. 233–241.

[27] L. Meng, Z. Ge, P. Tian, B. An, and Y. Gao, "An efficient deep reinforcement learning algorithm for solving imperfect information extensive-form games," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 5823–5831.

[28] T. Ren et al., "Enabling efficient scheduling in large-scale UAV-assisted mobile-edge computing via hierarchical reinforcement learning," *IEEE Internet Things J.*, vol. 9, no. 10, pp. 7095–7109, May 2022.

[29] H. Zhang et al., "Mean-field-aided multiagent reinforcement learning for resource allocation in vehicular networks," *IEEE Internet Things J.*, vol. 10, no. 3, pp. 2667–2679, Feb. 2023.

[30] D. Wang et al., "Mean field game-based waveform precoding design for mobile crowd integrated sensing, communication, and computation systems," 2024, arXiv: *2309.02645*.

[31] D. Wang, W. Wang, H. Gao, Z. Zhang, and Z. Han, "Delay-optimal computation offloading in large-scale multi-access edge computing using mean field game," *IEEE Trans. Wireless Commun.*, vol. 23, no. 3, pp. 1684–1698, Mar. 2024.

[32] C. Sun, W. Ni, and X. Wang, "Joint computation offloading and trajectory planning for UAV-assisted edge computing," *IEEE Trans. Wireless Commun.*, vol. 20, no. 8, pp. 5343–5358, Aug. 2021.

[33] Y. Deng et al., "AUCTION: Automated and quality-aware client selection framework for efficient federated learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 8, pp. 1996–2009, Aug. 2022.

[34] J. Xu and H. Wang, "Client selection and bandwidth allocation in wireless federated learning networks: A long-term perspective," *IEEE Trans. Wireless Commun.*, vol. 20, no. 2, pp. 1188–1200, Feb. 2021.

[35] Y. Yu et al., "Multi-objective optimization for UAV-assisted wireless powered IoT networks based on extended DDPG algorithm," *IEEE Trans. Commun.*, vol. 69, no. 9, pp. 6361–6374, Sep. 2021.

[36] S. Zhu, L. Gui, D. Zhao, N. Cheng, Q. Zhang, and X. Lang, "Learning-based computation offloading approaches in UAVs-assisted edge computing," *IEEE Trans. Veh. Technol*, vol. 70, no. 1, pp. 928–944, Jan. 2021.

[37] J. Ji, K. Zhu, C. Yi, and D. Niyato, "Energy consumption minimization in UAV-assisted mobile-edge computing systems: Joint resource allocation and trajectory design," *IEEE Internet Things J.*, vol. 8, no. 10, pp. 8570–8584, May 2021.

[38] H. E. Stanley, *Phase Transitions and Critical Phenomena*, vol. 9. Oxford, U.K.: Clarendon, 1971.

[39] P. Qin, Y. Fu, Y. Xie, K. Wu, X. Zhang, and X. Zhao, "Multi-agent learning-based optimal task offloading and UAV trajectory planning for agin-power IoT," *IEEE Trans. Commun.*, vol. 71, no. 7, pp. 4005–4017, Jul. 2023.

[40] X. Li, Y. Qin, J. Huo, and W. Huangfu, "Computation offloading and trajectory planning of multi-UAV-enabled MEC: A knowledge-assisted multiagent reinforcement learning approach," *IEEE Trans. Veh. Technol*, vol. 73, no. 5, pp. 7077–7088, May 2024.

[41] Z. Sun, Y. Mo, and C. Yu, "Graph-reinforcement-learning-based task offloading for multiaccess edge computing," *IEEE Internet Things J.*, vol. 10, no. 4, pp. 3138–3150, Feb. 2023.

[42] T. Pamuklu, A. Syed, W. S. Kennedy, and M. Erol-Kantarci, "Heterogeneous GNN-RL-based task offloading for UAV-aided smart agriculture," *IEEE Netw. Lett.*, vol. 5, no. 4, pp. 213–217, Dec. 2023.

[43] B. Xu, Z. Kuang, J. Gao, L. Zhao, and C. Wu, "Joint offloading decision and trajectory design for UAV-enabled edge computing with task dependency," *IEEE Trans. Wireless Commun.*, vol. 22, no. 8, pp. 5043–5055, Aug. 2023.

[44] Z. Kuang, H. Wang, J. Li, and F. Hou, "Utility-aware UAV deployment and task offloading in multi-UAV edge computing networks," *IEEE Internet Things J.*, vol. 11, no. 8, pp. 14755–14770, Apr. 2024.

[45] H. Shi, Y. Tian, H. Li, J. Huang, L. Shi, and Y. Zhou, "Task offloading and trajectory scheduling for UAV-enabled MEC networks: An MADRL algorithm with prioritized experience replay," *Ad Hoc Netw.*, vol. 154, 2024, Art. no. 103371.

[46] N. Zhao, Z. Ye, Y. Pei, Y.-C. Liang, and D. Niyato, "Multi-agent deep reinforcement learning for task offloading in UAV-assisted mobile edge computing," *IEEE Trans. Wireless Commun.*, vol. 21, no. 9, pp. 6949–6960, Sep. 2022.

Zhen Gao received the MS degree from the School of Computer Science and Engineering, Northeastern University, Shenyang, China, in 2020, and the PhD degree in computer science and technology from Northeastern University, China, in 2025. His research interests include MEC, task offloading, and reinforcement learning.

Gang Wang received the BS and MS degrees, in 2015 and 2019, respectively. He is currently working toward the PhD degree with the Department of Computer Science and Engineering, Northeast University, Shenyang. He research interests include blockchain and federated learning.

Lei Yang received the PhD degree in computer application and technology from Northeastern University, China, in 2008. He is an associate professor with the School of Computer Science and Engineering, Northeastern University, China. His current research interests include Big Data and cloud computing.

Yu Dai received the PhD degree in computer application and technology from Northeastern University, China, in 2008. She is an associate professor with the College of Software, Northeastern University, China. Her current research interests include Big Data, cloud computing, and intelligent web information system.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 08,2026 at 04:39:35 UTC from IEEE Xplore. Restrictions apply.